

Interface-Induced Trajectory Censoring

A mismatched serving interface distorts both what a benchmark measures
and what RL training ever sees

Wenbo Wang
City University of Hong Kong
wenbwang3-c@my.cityu.edu.hk

Di Sang
Beijing Institute of Technology
3120265429@bit.edu.cn

September 26, 2026

Code, data and pre-registrations:

<https://github.com/nebula-1999/Interface-Induced-Trajectory-Censoring>

Abstract

Tool-use evaluations typically measure calls through a serving stack, which can report zero calls even when model outputs contain call-shaped payloads. We study interface-induced trajectory censoring: a mismatch between model emissions and the interface that converts them into actions. On selected BFCL v4 subsets, changing only the serving adapter while holding model weights and other evaluation settings fixed moves scores under the official scorer from 0.00 to 0.96 on `simple_python` and from 0.00 to 0.19 on `multi_turn_base`. A template-parser factorial experiment shows strong complementarity: neither component replacement alone restores parsing from the baseline, whereas their joint replacement recovers 196/200 first-request parses. Across five Qwen2.5-Coder checkpoints, classifier-positive emissions increase with model size while the mismatched interface reports no calls; a separate matched-interface Qwen3 ladder exhibits few silent emissions through 8B. Experiments on 115 τ -bench retail tasks extend the mechanism to a stateful, non-code environment. In a preregistered, single-seed, 150-step 7B LoRA experiment, changing only the parser registered in verl increases tool executions from 10 to 16,844. However, neither a shared programmatic-feedback evaluation nor a post-hoc common native function-calling evaluation detects a held-out success gain from repair; rescue counts do not increase. Interface repair thus restores tool-mediated experience without a detected performance gain in this setting. These findings show that tool-use measurements depend on the model-interface configuration and motivate layer-wise observability and conformance checks before attributing evaluation or training outcomes to the model alone.

1 Introduction

Tool-use evaluation observes a model through a serving interface. Under selected BFCL v4 subsets, the same model receives official scores of 0.00 or 0.96 / 0.19 depending on its serving adapter. We investigate how serialization contracts alter observed actions and the experience available for training. Historical 1.5B runs motivated the investigation, but their non-significant gains and ambiguous zero-execution mechanism are supporting observations, not the basis of the formal 7B result (Appendix G).

An agent evaluation does not measure a model. It measures a composition `model` $\times$ `protocol` $\times$ `serialization` $\times$ `parser` $\times$ `execution stack`, and the failure of most stages is indistinguishable at the output from model incapability: when a parser does not recognise the format a model

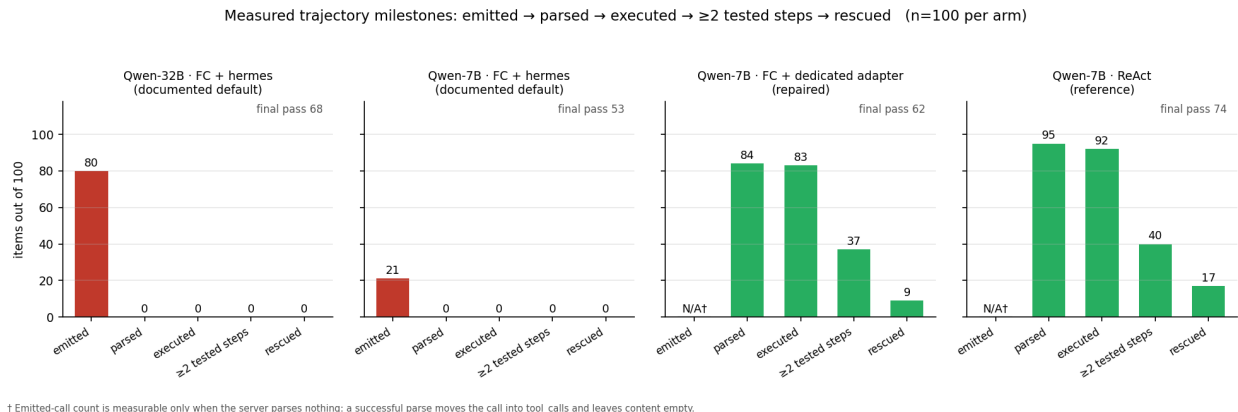


Figure 1: **Measured tool-use milestones.** $n=100$ per arm. The two left panels show classifier-positive call-shaped emissions but no parsed tool calls. The fourth bar counts tasks reaching at least two code-bearing tested steps (37 repaired; 40 ReAct), not returned observations; the decline from executions to second tests does not measure feedback-return failures. Emitted-call counts are measurable only where the server parses nothing — a successful parse moves the call into `tool_calls` and empties `content` — so those cells are N/A, not zero.

emits, the serving layer reports zero tool calls — exactly what a model that refuses to use tools would produce. We call this **interface-induced trajectory censoring**.¹ It acts in two opposite directions, and our results contain a clean instance of each: **masking**, where a valid call is not recognised and the action never reaches the environment (§4.3), and **suppression**, where an invalid call is removed from the sampleable support before it becomes an action (§4.5). **The serving interface is therefore simultaneously part of the measurement function and part of the agent’s effective action space** — literally so under RL, where whatever the policy emits must survive serialisation and parsing before it becomes an action that earns a reward. When nothing tool-intended survives that passage, the experience distribution contains no tool-mediated trajectory at all.

Why this is not a parser bug. The term invites a reading we refuse at the outset. Replaying vLLM’s own `hermes` extractor line by line over every stored first turn in our archive — 4254 of them, after removing byte-identical duplicate files — we find zero cases in which the server failed to parse a call its own rules would have accepted (§4.3). What fails is the *contract* between three parties configured independently: what the checkpoint was trained to emit, what the chat template tells it to emit, and what the parser is defined to accept. Qwen2.5-Coder emits a well-formed bare call and never the `<tool_call>` envelope, so `hermes` not seeing it is *correct behaviour*; Qwen2.5-Instruct emits the envelope and then breaks the JSON inside it. Censoring can occur at any boundary along `emission` → `serialization` → `parse` → `execution`, and at every one the observable is identical: HTTP 200 with an empty `tool_calls` array. The claim is therefore not that parsers are buggy but that **a tool-call rate is a property of a three-way contract, and is routinely reported as though it were a property of the model alone.**

¹Borrowed by analogy: in survival analysis censoring is a threshold on a continuous variable, whereas a parser is a deterministic filter on output *format*. What carries over is the structure of the inference error — the observation is not missing at random but systematically unobservable on one side of a boundary, and the boundary correlates with the quantity being measured.

What is and is not new. That Qwen2.5-Coder does not emit `hermes-format` tool calls was reported in vLLM issue #32926 (2026-01-23) with a `<tools>` parser proposal; it was closed as *not planned* and labelled *stale*, so the trap is still live (§2). What this paper adds is measurement, decomposition and consequence. We measure the funnel on *two* standard benchmarks — BFCL v4 and τ -bench — where changing only the serving adapter moves BFCL’s reported score from 0.00 to 0.96 / 0.19 and τ -bench’s parsed calls from 0 to 636, and a 2×2 over template and parser finds zero recovery from either one-component replacement at the documented baseline and 196/200 parses when both are replaced (§4.1). We separate *model intent* / *emitted format* / *server parse* / *actual execution* / *multi-turn rescue* into five independently measured quantities — reporting only the third is the practice we argue against — quantify what censoring removes across a $21\times$ scale range, and pair that ladder with a preregistered supporting ladder under a *matched* envelope where the silent fraction stays at 0–2 instead of rising to 80 (§4.3). Four families failing at template, parser, schema and token layers motivate a layered taxonomy on deliberately asymmetric evidence (§4.6). And we follow the distortion into training. A preregistered 150-step 7B LoRA control changes only the parser registered inside verl: the mismatched arm executes 10 calls, versus 16,844 under the repaired parser, while held-out rescues are 12 at both endpoints in both arms. The intervention therefore isolates a large change in the sampled experience distribution without turning that mechanistic repair into a learning claim (§4.8).

2 Related Work

Evaluation validity under nuisance factors. Reported LLM capability is sensitive to factors that ought to be irrelevant: prompt formatting alone moves benchmark scores by margins comparable to model differences [Sclar et al., 2024], single-prompt evaluation is unreliable [Mizrahi et al., 2024], and benchmark and configuration choices determine which method appears to win [Dehghani et al., 2021]. Those works vary an input the evaluator controls and observe score movement. **We identify a component of the measurement apparatus — the serving stack’s tool-call parser — that can zero out a capability signal entirely, without any error, and whose distortion grows with model scale.**

Harness-Bench measures configuration-level harness effects across realistic workflows and argues for reporting model–harness configurations [Yao et al., 2026]. We instead isolate a serialization–parser contract with component-level interventions and trace it into RL experience. Agent-Reactive Bugs finds silent model–harness boundary failures in 255 issue reports [Chen et al., 2026]; our evidence comes from controlled paired interventions rather than a bug-report taxonomy.

Parsing-induced measurement error. The mechanism has been isolated once before, in another domain: Garware and Zisad [2026] show that in LLM-based security-log classification a strict regular-expression parser reports 0% threat accuracy while a corrected fuzzy parser recovers 76% on *identical* model outputs, with an unaffected severity metric as an internal control. We regard it as the closest antecedent and adopt the same standard of proof: one fixed set of bytes, reparsed under different rules (§4.3). Three things separate the contributions — their gap is a single mechanism in a single domain, which they explicitly decline to generalise, whereas we separate four layers across four families; their gap is a fixed quantity, ours is a function of scale; and their distortion terminates at the reported number, whereas we follow it into an RL rollout and identify which branch of the policy the gradient can no longer reach (§4.8). Outside the literature the claim that tool-call failures originate in serving infrastructure is already practitioner folklore [LiveKit, 2026], qualitatively and without measurements.

Audits and failure taxonomies of tool use. A cluster of concurrent 2026 work asks whether

reported tool-use scores mean what they appear to mean, and stops one layer above us: Bhat et al. [2026] report an 18.5% evaluator–human misalignment rate across BFCL v4, τ^2 -bench [Barres et al., 2025], LiveMCPBench and MCP-Atlas; τ^2 -Bench-Verified [Amazon AGI, 2025] documents four categories of ground-truth defect in a benchmark used to rank frontier agents; Raj et al. [2026] organise 41 failure modes by interaction edge and by whether the repair belongs to model or harness; Soni [2026] label traces across 1,000 tasks as Tool-Skip, Result-Ignore, Output-Fabrication or Unnecessary-Tool-Use; and Albayaydh et al. [2026] synthesise 27 such papers into six clusters, one being measurement validity. In two cases our result is a precondition for theirs: *Model or Harness?* supplies the vocabulary but no measurement, and *ToolFailBench*’s Tool-Skip label is read as a model behaviour when it is also exactly what an unparsed emission produces — the category is confounded by the interface unless the stack is verified first, which the preflight check of §5 does in seconds. **Every audit above varies something above the API boundary; the distortion we report occurs below it, and is invisible to all of them.**

Constrained decoding, protocols, stacks. Tam et al. [2024] report that format restrictions can degrade reasoning; §4.5 is a counter-instance, where `strict: true` reduces wrong-tool calls from 22/100 to 0 under the official template and *improves* pass rate, though we also report the CoT control where added reasoning made matters worse. Guided-decoding machinery [Willard and Louf, 2023] underlies both `strict` and vLLM’s `required` mode. ReAct [Yao et al., 2023] is the text protocol we compare against, and BFCL [Patil et al., 2025], τ -bench [Yao et al., 2025], ToolLLM [Qin et al., 2024], API-Bank [Li et al., 2023] and Toolformer [Schick et al., 2023] evaluate or train tool use, reporting tool-call rates as parsed by the serving layer. verl [Sheng et al., 2025] provides the RL stack, and importantly its AgentLoop performs **its own** tool-call parsing (`verl/experimental/agent_loop/tool_parser.py`, adapted from vLLM v0.9.1), so a vLLM-side parser plugin does not affect training rollouts — a distinction that cost us one wasted experiment design (§4.8).

Prior report of the Qwen case. vLLM issue #32926 [hanXen, 2026], opened 2026-01-23, documents that Qwen2.5-Coder was not trained on tool-call tokens, that it emits `json` fenced blocks or bare JSON absent format instructions, and that a `<tools>` few-shot template plus a dedicated parser reaches 98–100% compliance. It was closed as *not planned* and labelled *stale*. §4.3 reproduces this baseline exactly — zero `<tool_call>` and `<tools>` tags at every scale — and extends it to scale quantification, three further families and training; a related verl issue [ver, 2025] reports the same role-confusion signature we characterise in §4.5. **Our claim is not that the interface can fail silently, which is known; it is that the silent fraction is measurable, that it grows with checkpoint size under the same mismatched interface, and that it propagates into training.** A literature search conducted 2026-09-01 returned nothing that follows the distortion below the API boundary into training; we state this as the scope of our search rather than as a claim of novelty.

3 Setup

Task. 100 problems from a decontaminated KodCode [Xu et al., 2025] subset (n-gram containment against EvalPlus [Liu et al., 2023]). All 50 full-length arms use the identical 100 items (verified: unique item-set count = 1), so every cross-arm comparison is paired.

Protocols. *ReAct* — a text protocol with Thought: / Action: `run_tests` / Action Input: / Observation:, tool name fixed in the template. *Function calling (FC)* — OpenAI-style, `tool_choice: auto`, one tool `run_tests` taking a single code string, in three schema variants: terse (bare `{"type": "string"}`), rich (description, counter-example, code sample), strict (`strict:`

`true, additionalProperties: false).`

We train with GRPO [Shao et al., 2024] and RLOO [Ahmadian et al., 2024]; the formal 7B control uses matched rank-32 LoRA arms [Hu et al., 2022].

Serving. vLLM 0.27.1 [Kwon et al., 2023, vll, 2026], per-family parser and chat template as documented. `temperature=0` is greedy but not bit-wise deterministic under continuous batching; the main results are a single sample per item and Appendix G quantifies sampling variability separately at `temperature=0.6`. `max_tokens = 2048` for both protocols (Appendix A: an earlier asymmetry favoured FC; re-running under the true shared limit changed final pass by ≤ 1 item). Serving flags per family and the full training configuration are in Appendix D; every arm’s recorded configuration is in Appendix E.

3.1 Intent criterion, and its validation

Reporting only server-parsed calls is the practice under examination, so we need an independent measure of what the model emitted. For the KodCode probe, a shared classifier (`analysis/intent.py`) is shared by all text, tables and figures, with three operational tiers: `tight` is a regex match for a call-shaped payload naming `run_tests`, with quoted `arguments.code` and Python-like lexical markers. It does not parse JSON or establish executable Python, though it excludes some schema echoes and unquoted placeholders. Strong uses the trajectory tags `json_named_call` or `xml_tool_call`; weak uses tags for JSON-shaped fragments not naming the tool, which at 1.5B are almost entirely false positives and are reported separately rather than merged. Every prompt, schema and tier definition is reproduced verbatim in Appendix C.

Three facts about the validation carry into the results; the full two-round procedure, the instrument failure that produced the first round’s $\kappa = 0.713$, the adjudication rule and the three successive values of the correction factor are in Appendix F. **(i)** Against a gold standard built by third-party adjudication of all fifteen non-unanimous items — selected by *any* three-way disagreement, never by disagreement with the classifier — the classifier scores $\kappa = 0.871$ [0.756, 0.958], and two blind annotators reach inter-annotator $\kappa = 0.967$ on the 62 items where both read identical bytes. **(ii)** The `tight` stratum’s precision against that gold standard is $36/40 = 0.900$, so §4.3’s headline 80/100 at 32B gives a sensitivity estimate of ≈ 72 under transferable precision. It does not recover false negatives or estimate all true calls; applying a constant factor preserves the trend by construction, not by independently validating scale-specific precision. **(iii)** The probe stores at most 4000 characters of raw output per item and 10 of the 98 sampled outputs reach that ceiling, creating downward pressure from false negatives. Since precision is 0.900 and thus also reveals false positives, the net bias is not identified; we report raw counts and the sensitivity estimate rather than calling the total a strict lower bound.

3.2 Positive control: the pipeline works

Before attributing a zero to a model we verify the instrument. On the same live server, model, tools and prompt, changing only `tool_choice`: under `auto` the server returns `finish_reason = stop` with `tool_calls = 0` while the assistant writes the solution as fenced code; under `required` it returns one parsed call naming `run_tests` with real Python in `arguments`. Server flag, parser and template are therefore all functional. **This establishes that the pipeline can accept a conforming call; it does not locate the zero under auto in the model**, since `required` uses constrained decoding and says nothing about whether the model spontaneously knows the format. The emitted-intent measurement of §4 is what separates them.

3.3 Validity gating

An arm is admissible for pass-rate comparison only if it has exactly 100 items, exit code 0, **zero request errors**, and provenance (model / protocol / temperature / **max_tokens** / adapter) matching the intended configuration. Arms with request errors retain a valid error-rate census but are marked N/A for pass rates and *no p-value is computed for them*. **This rule is enforced in code** (`validate_arms.py`, `final_table.py`), not by convention: an earlier draft computed $p=0.648$ on such an arm, and that number is retracted (Appendix A).

For paired binary outcomes we use exact two-sided McNemar tests and report both discordant counts. For the formal RL endpoint we report a conservative finite-sample 95% paired-risk interval from exact binomial bounds on gain/loss probabilities. It describes item-pair uncertainty, not training-seed or inference-retest variation. Appendix I enumerates a 27-test retrospective family, with Bonferroni threshold $0.05/27 \approx 0.00185$; this reporting policy was not preregistered. Historical motivating runs remain exploratory.

Historical 1.5B curves exclude two verified EvalPlus defects (`HumanEval/32` and `Mbpp/599`) and use $n = 540$. The preregistered 7B control retains all 542 items and counts those two as failures in both arms. They lower absolute rates but do not enter the paired discordances used for the arm comparison.

4 Results

Nine measurements, in the order the argument needs them. Each is stated here with its numbers; the tables, the funnel columns behind them and the reproduction records are in Appendix G, and every scope condition is collected in §6.

4.1 BFCL v4: the score moves the whole span of the metric, and repair is complementary

BFCL v4 supplies official data, a multi-turn executor and a scorer at pinned Gorilla commit `6ea57973c7a6097fd7c5915698c54c17c5b1b6c8`. Holding weights, cases, decoding, seeds, executor and scorer fixed on 200 cases and changing *only the serving adapter configuration*, BFCL’s own scorer reports **0.00** or **0.96 / 0.19** (`simple_python` / `multi_turn_base`) for the same model. The intervention is a pair — parser *and* its chat template — so we ran the 2×2 that separates them.

Table 1: Template $\times$ parser on BFCL, first request per case.

	documented template	dedicated template
hermes parser	0/200	0/200
dedicated parser	0/200	196/200

Starting from the documented configuration, either one-component replacement recovers 0/200 calls, whereas replacing both components recovers 196/200. These are simple effects at a specified baseline, not zero marginal main effects. The 2×2 establishes strong complementarity in parsing; BFCL scores were computed only for the two end-to-end corner configurations, so the score swing itself is not decomposed over all four cells. The templates induce different output conventions; the experiment does not hold payload bytes fixed across templates. The two one-component substitutions recover no calls at this baseline. The funnel behind those scores: the documented arm parses **0 of 200** first requests with zero HTTP errors, while **166/200**

responses satisfy the **BFCL JSON-call criterion**; the repaired arm parses 196, executes 98 of 100 `multi_turn_base` cases and succeeds on **19**, against 0 executed and 0 succeeded for **hermes**, whose zero is therefore *by construction rather than by capability* (Tables 10–20). **A published benchmark number for this model can differ by the entire span of the metric depending on a serving flag the model card does not mention.** We claim only that 0.00 is not a measurement of the model; and since the repaired arm uses the community adapter, not a verified-optimal one, and still fails 4 of 200. We therefore report $0 \rightarrow 19$ as the recovery achieved by this adapter, not as an estimate of an optimal model or interface score.

BFCL’s shipped handler for local Qwen models bypasses the serving parser entirely, so we route BFCL’s own OpenAI handler through `/v1/chat/completions`, adding only `tool_choice: auto` and trace headers; dataset loading, execution and scoring remain BFCL’s code and `bfcl_eval`’s files are byte-identical (Appendix G.4).

“Documented” denotes application of Qwen2.5 family-level guidance, not a Coder-specific compatibility guarantee (Appendix D). In a separate answer-blind offline analysis, a permissive JSON extractor recovers 97 structurally schema-compliant calls from the original 100 `simple_python` responses; 92 pass the pinned official AST checker without changing the emitted bytes. This extractor is not the deployed dedicated parser, which requires `<tools>` tags absent from all 100 responses (Appendix G.4).

4.2 τ -bench: the mechanism survives a simulated user and a stateful environment

BFCL scores a single trajectory against a final state. τ -bench [Yao et al., 2025] adds a simulated user and a stateful environment, so a censored call costs not one action but the rest of the conversation. Across all 115 **retail** tasks, paired across arms with none dropped and changing only the serving adapter: server-parsed calls **0 versus 636**, tool executions **0 versus 636**, and tasks reaching *any* tool execution **0 versus 103**. Every one of the documented arm’s 1389 requests carried `tools`, not one was parsed, and 789 assistant turns carried a call-shaped payload the server discarded (Table 11). **The task-outcome difference does not reach significance and we do not claim it:** 7 versus 10 solved, three discordant pairs all in the repaired direction, exact McNemar $p=0.25$, with the documented arm’s solved set a strict subset of the repaired arm’s. Two scope conditions belong with these numbers rather than in a footnote: the user simulator is a locally served Llama-3.1-8B rather than the `gpt-4o` of the original harness, so **the absolute scores are not comparable to the τ -bench leaderboard** and only the between-arm difference is claimed; and the arms differ in conversation length by construction, which is why every quantity is per task and not per request.

The missing-envelope signature also appears in a non-code domain. τ -bench also lets us ask whether the *failure layer* travels across domains, because its arguments are short scalars (`{"order_id": "W123"}`) where our own task passes a whole program as a string. Replaying vLLM’s extractor over the documented arm’s 789 emitted-but-unparsed assistant turns assigns **789 of 789 (100%) to the missing-envelope category**. None reaches the payload-validation stage, and none is accepted by the configured extractor. This first-failure classification establishes neither payload validity nor recoverability under a different decoding rule. It reproduces the missing-envelope signature in one non-code domain, not domain invariance. For comparison, Qwen2.5-Instruct has 41% of its unparsed code-task items assigned to malformed payloads after the envelope check (Table 19). Because the τ -bench items do not reach that check, these data do not identify

a cross-domain difference in payload validity. Whether payload-error proportions change with argument type remains an open question.

4.3 Censoring against scale, and the same scale under a matched interface

Our own probe, across the Qwen2.5-Coder ladder under `tool_choice: auto` and `hermes` — the documented recommendation for Qwen2.5 — $n=100$ per size.

Table 2: Qwen2.5-Coder under the documented `hermes` configuration, $n=100$ per size.

Size	Server-parsed	tight	strong labels	weak labels	Final pass
1.5B	0	0	0	27*	31
3B	0	4	5	28	37
7B	0	21	22	1	53
14B	0	36	42	3	53
32B	0	80	100	0	68

Across a $21\times$ range the server reports a flat zero while well-formed emitted calls rise to 80/100 (≈ 72 after the 0.900 correction factor of §3.1, assuming tight-stratum precision transfers across sizes). **Within this family, the larger the checkpoint, the larger the absolute undercount under the same mismatched interface** — a within-family observation across five checkpoints, not a law relating capability to censoring. Raw-tag counts reproduce vLLM issue #32926’s baseline exactly: `<tool_call>` and `<tools>` appear zero times at every size. Two offline analyses close the obvious objections (Appendix G.3): re-parsing the *same stored bytes* under four rules excludes “your extractor is simply better than `hermes`”, and replaying vLLM’s extractor line for line to locate each loss shows **genuine parser loss is zero in every arm**, across all 4254 de-duplicated first turns.²

A matched-interface supporting ladder through 8B. The server column is flat zero, so the growth comes entirely from the emitted column — and larger checkpoints emit more well-formed JSON for reasons unrelated to tool calling. Qwen3 supplies a ladder whose template injects tools and requires the `<tool_call>` envelope `hermes` accepts; the prediction (“parsed > 0 at every size”) was committed to the repository before running (p4/PREREGISTRATION.md). Server-parsed counts are 86 / 44 / 70 / 97 at 0.6B–8B, and the silent fraction through 8B is **1, 1, 2, 0** against the mismatched ladder’s 0, 4, 21, 36, 80 (Table 12). **Across a comparable span of scale under a matched envelope, we do not observe the growing silent fraction seen in Qwen2.5-Coder.** This is a cross-family supporting observation, not a strict scale counterfactual: it changes family and envelope together and does not independently measure capability growth. Two caveats we state rather than bury: the parsed column is **not monotone**, so this shows presence, not a trend; and Qwen3 was served with `enable_thinking=false`, because with thinking on the model exhausts its token budget in a `<think>` block before emitting a call — a confound unrelated to the envelope, recorded as a protocol deviation before the run.

A second family showing the *same* undercount would be stronger, and we did not get one. Of nine families surveyed beforehand, six do not inject OpenAI-style tools at all; of the remaining three, Granite-3.1-8B does not attempt tool calls on this task — 0 under `hermes` and 0 under its own `granite` parser. **That rescue arm is what tells us the arm is uninformative rather than confirmatory**, and we claim nothing from it.

²The 1.5B row uses the *optional* tool instruction; under the *mandatory* prompt used for training (§4.8) the same checkpoint gives weak = 66 and unparsable = 46. Coercion produces more JSON-shaped debris, not more valid calls.

4.4 Within-lineage control, and the ceiling it puts on multi-turn rescue

Changing family cannot test the scale result — Llama parses correctly, Mistral returns 400, DeepSeek never injects the template, so none can exhibit censoring at all. Qwen2.5-**Instruct** can: it shares Coder’s chat template, which is what produces the false-positive template check of §4.6, and differs in whether the checkpoint was trained on tool-call tokens. On the same items, parser, `tool_choice`, temperature and seed, its server-parsed rate runs **1, 11, 63, 77, 88** against Coder’s flat zero (Table 13). **The contrast localises the mismatch to checkpoint-specific training rather than to the shared family and template, or to parameter count alone.** We stop short of naming tool-call-format training as *the* cause: the two differ in their whole post-training mixture, and this varies the bundle, not one element. It also answers the objection that Instruct emits better-shaped calls around worse contents — it solves 13–42% of items on turn 1 and 13–65% by the end, nowhere near the zero that would require.

The multi-turn layer was initially void, because that host lacked `pytest`; we re-ran all five arms with the executor working, changing nothing else and not touching the probe binary, and the parsed column reproduced value for value. **Rescues then rise with the parsed column** — 0, +3, +6, +14, +23 against 1, 11, 63, 77, 88, monotone together. Coder’s first-turn and final passes are identical at every size (31/31, 37/37, 53/53, 53/53, 68/68), which it would be natural to read as multi-turn repair being useless at this scale; the Instruct ladder, on the same items and criterion, shows it is not. We therefore state the repair result as a ceiling rather than an absence: **how much multi-turn recovers is bounded by how much of the tool channel the interface leaves open.**

4.5 A 23% failure that survived three controls and fell to the fourth

Llama-3.1-8B under FC issues a tool call on 97/100 items, but on 23 the call names *the function the task asks it to implement*, with that function’s own parameters as arguments; a re-run restricted to those 23 reproduced 23/23. *This also corrects a measurement of our own*: because the probe did not verify the tool name, these 23 were counted as initiations, so **Llama’s true run_tests initiation rate is 74/100, not 97/100** (Appendix A). Four single-variable paired controls: a richer schema explicitly warning that the tool is not the task function left it at 22/100 ($p=1.000$); a Thought scaffold before the call raised it to **59** ($p<0.001$, *worse*), the model supplying exactly the parameters it had just reasoned about; the official chat template moved nothing (23→22, wrong-tool discordances 1/0, exact $p=1.000$); **strict: true** on the official-template baseline took it from 22 to **0** (22/0, exact $p=4.77\times 10^{-7}$). The earlier 0.125 and 0.0001 labels came from different outcome comparisons and are withdrawn from the wrong-tool table. The first three rejected their alternative explanations and pointed at the model; the fourth found the cause and withdrew that conclusion.

Adding **strict** to the official-template arm drives wrong-tool calls from 22 to 0, valid execution from 73 to 97, turn-1 pass from 33 to 46, and final pass from 44 to 61. Thus the single-variable final change is +17 (Table 14). The ReAct decomposition instead uses terse FC (49) as its baseline and official-template-plus-strict (61) as the repaired endpoint, so its broader interface-bundle contribution is **+12** ($p=0.0075$), followed by **protocol, +19** ($p=0.0019$), against +31 for both (Table 15). The +12 and +19 contrasts are descriptive: neither survives the retrospective family-wise correction. The official-template strict final-pass comparison does survive that correction. With **strict** the two protocols have identical execution mechanics (98 calls, 0 wrong-tool, 97 executed on both sides) yet turn-1 differs 46 vs 61, so what remains is first-draft code quality under the protocol, not tool-use mechanics. The defensible statement is narrower than either “model defect” or “configuration bug”: **the model exhibits role confusion under unconstrained function calling,**

and the interface constraint determines whether that confusion can be expressed as an environment action. We cannot show the tendency disappears once constrained, only that it stops reaching the action space.

4.6 Four layers, and what a pre-flight check can see

The two families above fail at different layers with different remedies — Qwen2.5-Coder at the *parser* layer, repaired by a dedicated adapter; Llama at the *schema* layer, repaired by `strict: true` — and neither is caught by the obvious pre-flight test. Two further families extend the taxonomy without supporting a quantitative comparison, and we report them as such. **DeepSeek-Coder** fails at the *template* layer: its chat template never injects tools, which a template check does catch, and we found no remedy. **Mistral-7B-v0.3** fails at the *token* layer, repeating a `[TOOL_CALLS]` marker that produces HTTP 400, and has **no admissible 100-item FC run at all** — all four configurations produce request errors (parser only 2; official template 42; parser + strict 3; official template + strict 39), so vLLM’s own recommended configuration *raises* the error rate from 2% to 42% and `strict`, which fixes Llama, does not help (Table 21).

Mistral is the one family whose failure announces itself; the other three return HTTP 200 with `tool_calls: []`, byte-identical to a model that declined to call the tool. **The claim is not that all failures are silent; it is that the silent ones dominate and are the hardest to attribute.** The natural pre-flight test — does `apply_chat_template(tools=...)` render the schema? — correctly rejects DeepSeek-Coder but *passes* Qwen2.5-Coder, which inherits Hermes scaffolding from Qwen2.5-Instruct while never having been trained on it: **protocol support is a per-checkpoint property and is not inferable from the chat template.**

4.7 Repair loop: how much comes back

On Qwen2.5-Coder-7B with `tool_choice: auto` held fixed and only the adapter changed, parse goes **0→84**, items reaching a second turn **0→37** and multi-turn rescues **0→9**: the mechanism is restored from literal zero (Table 16). The two FC arms have identical aggregate turn-1 pass, 53 vs 53. Because the evaluation adapter changes parser, template and few-shot together, equality of this aggregate does not establish identical first-turn generations and cannot by itself locate the failure layer. Llama’s generation-time constraint does move turn-1 pass on the official template (33→46), but we use fixed-output replay and per-event tracing, not the direction of aggregate score movement, to localize both mechanisms.

The pass-rate gain is *not* significant (53→62, $p=0.093$). A pre-planned replication on a random 300-item sample reproduces the direction (160/300→178/300, 43 discordant against 25, $p=0.0385$) — nominally below 0.05, but the explicit retrospective family contains 27 tests and its Bonferroni threshold is ≈ 0.00185 , so **the repair gain still does not survive correction and we rest no claim on it.** The protocol gap on the same 300 items (178/300→219/300, $p=3.1\times 10^{-6}$) clears that threshold. Restricting to items both arms parsed, the descriptive residual on Llama is nominally significant ($p=0.0021$) but does not clear that threshold and is not detected on Qwen: +8.4 pp, 95% CI $[-0.5, +17.4]$, $p=0.118$. **We do not read $p > 0.05$ as evidence of no effect** — that interval admits a residual larger than the effect we detect on Llama — so the residual is established on one family and undetermined on the other (Appendix G.8).

4.8 Parser repair restores RL experience; no held-out improvement detected

The historical 1.5B result is over-determined: zero calls executed, but a replication under the recorded mandatory-prompt override finds no correctly named call; exact historical prompt bytes

are unavailable (Appendix E). At 7B, by contrast, a probe inside verl’s AgentLoop records 45 complete bare-JSON calls in 115 generations, of which **hermes** accepts none; its actor update then OOMs, so that probe establishes the failure layer but not a learning effect. We therefore ran the missing control: two preregistered 150-step GRPO arms at 7B under identical LoRA configuration, data, prompt, seed, decoding, verifier and evaluation. The sole intervention is `multi_turn.format`: verl’s stock **hermes** parser versus a registered parser that accepts Qwen2.5-Coder’s emitted format. Full rollout text, parser verdicts, executions and returned observations were retained.

Table 3: Formal 7B RL control. Tight counts generation events, accepted counts parsed calls, and executed/observed counts tool invocations. Training counts exclude held-out evaluation; outcomes count items in the common 542-item split.

arm	tight emissions	accepted	executed/observed	final, step 0→150	rescues, 0→150
broken FC	8,689	10	10	430→430	12→12
repaired FC	17,608	16,912	16,844	430→427	12→12

The 16,912 accepted calls occur in 16,844 assistant-generation events: 40 generations contain multiple calls and contribute 68 calls beyond the first. With `max_parallel_calls=1`, the AgentLoop selects the first parser-returned call; additional calls are left undispached, not merged into one invocation. All 16,844 dispatches join to executions and observations; extraction-to-dispatch counts are matched per trajectory, not through a per-call identity. The tight classifier and parser acceptance are not nested sets, so their marginal totals should not be read as a sequential drop of 696 calls; Appendix G.10 gives the event-level cross-tabulation.

The broken channel is therefore *nearly*, not literally, closed: 8,689 tight-classified generation events and only 10 executions are recorded. Parser repair changes the sampled experience by more than three orders of magnitude and returns an observation for every executed call, while holding the learning problem fixed. This is direct causal evidence that the interface determines which emitted behaviour enters RL experience; it supersedes the earlier, confounded ReAct comparison for that claim.

Under a shared parser-independent evaluation that automatically executes generated code and returns test feedback, no held-out outcome improvement is detected. Repaired FC ends at 415/542 turn-1 passes and 427/542 final passes, versus 418/542 and 430/542 for broken FC; all three missing final passes are discordant in the repaired-to-broken direction (paired difference -0.55 percentage points; conservative paired 95% CI $[-1.79, 0.72]$; exact McNemar $p = 0.25$). Relative to its own step 0, repaired FC has one gain and four losses ($p = 0.375$), and multi-turn rescues are 12 at initialization and at the endpoint, with variation in between. We do not read either non-significant difference as harm. **At this model scale, 150-step budget and single seed, repairing the parser restores tool-mediated experience, but we detect no improvement in held-out success or rescue counts.** The earlier 75-step ReAct arm points in the same direction but is now only supporting evidence, because it changes protocol as well as interface.

Both arms use LoRA rank 32 because 7B full-parameter RL does not fit a single 80GB card. The held-out evaluation is identical across arms and bypasses both training parsers: verl’s native rollout RPC queries the current LoRA at temperature 0 for up to four 1,024-token programmatic-feedback turns on the same 542 EvalPlus items, then appends sandboxed test output as a user message. Each request attests the loaded global weight step, explaining why the broken-training arm can still exhibit 12 evaluation rescues. The shared evaluation deliberately avoids both training parsers, but changes structured tool-role feedback to user-message feedback. It measures transfer to this programmatic-feedback protocol, not native FC behavior. Formatting shift, limited adaptation

budget/capacity and multi-turn credit assignment are plausible explanations for absent transfer; these data do not separate them. The evaluator extracts the last decodable `<tool_call>` code argument when present, otherwise the last code fence or the raw text. This is not a general decoder for every tool syntax.

A separate post-hoc common native FC evaluation now measures voluntary initiation with the same working parser across base and both endpoint weights. Final successes are 239/542, 248/542 and 248/542; repaired versus broken has 16 discordances in each direction (0-point difference; conservative paired 95% CI $[-3.47, 3.47]$; $p = 1$). Initiation counts are 46/50/47 and rescues 2/1/1. Thus neither shared evaluation detects a success gain from repair; equal totals do not establish equivalent policies. Appendix G.15 gives the protocol, timeout audit and full outcomes.

The repaired process was interrupted after step 116 and resumed from the step-90 checkpoint; before resumption we verified model, optimizer, scheduler, RNG and dataloader state, source and configuration hashes, and native checkpoint weight steps. The repeated step-90 evaluation differs by two final-pass items (428 versus 426), which we report as test–retest variation; the formal curve uses the original step-90 result and the weight-attested resumed step-120/150 results. Mechanism totals combine original steps 1–90 with resumed steps 91–150 and exclude the abandoned 26-update step-91–116 branch. Appendix G.10 reports every checkpoint, training-log diagnostics, checkpoint tensor deltas, and the machine-verifiable final-lineage manifest.

5 Discussion

Report the composition, not the model. Any claim of the form “model M does not use tools” is, as measured, a claim about $M \times \text{protocol} \times \text{serialization} \times \text{parser} \times \text{stack}$. Agent evaluations should report at minimum (i) server-parsed calls, (ii) emitted-but-unparsed calls under a stated criterion, (iii) request errors, and (iv) the exact serving configuration. Reporting only (i) is the practice this paper argues against, and (ii) is where the $21\times$ scale trend lives.

Pre-flight, don’t post-hoc. A missing `--enable-auto-tool-choice` is the only failure that announces itself, and template inspection alone is insufficient (§4.6). We therefore treat a `tools`-bearing request returning HTTP 200 — *and* a server-parsed call under `tool_choice: required` — as a required pre-flight for any FC experiment, shipped as a 98-line script (`preflight_toolcall.py`) that issues one canonical request, asserts `tool_calls` is non-empty with the expected `name` and parseable `arguments`, then repeats under `required` as a positive control. This screening check flags configurations that fail to return an expected tool call; it does not by itself identify the underlying failure mechanism.

Generation constraints are a distinct potential remedy: they change the emission distribution rather than accept already emitted bytes. Our strict-schema control supports studying this option, not dismissing it. Syntax or allowed-name constraints alone do not establish correct arguments, successful task behavior or compatibility of the entire serving stack.

The guidance for this flag lives in the wrong place. We surveyed the HuggingFace model cards of nine families for any mention of `--tool-call-parser`. Of the eight reachable cards, none mentions it, while six recommend serving the model with vLLM. The publisher tells you which server to use, the server asks you which parser to use, and neither treats the pairing as its responsibility. We do *not* read this as evidence that mismatch is common in deployment — we surveyed documents, not deployments, and vLLM ships dedicated parsers for some forty families, so following *its* table generally works. The claim is narrower: **the one piece of configuration that can silently zero a capability measurement is documented by neither party at the point of use.**

Interface repair precedes protocol comparison. On Llama the ReAct-vs-FC gap was 31 points before the `strict` fix and 19 after, both component contrasts descriptive under the retrospective multiplicity correction. On Qwen, conditioning on parsed items, we do not detect a residual ($p=0.118$) — but that interval is wide enough to contain effects as large as the unconditioned gap, so it is evidence of *insufficient resolution*, not equivalence. **Comparisons of agent protocols that do not first exhaust the serving configuration matrix are not measuring protocols.**

A cold-start hazard for agentic RL. In the formal control, the mismatched parser admits only 10 executions from 8,689 tight-classified call-like emissions while a tool-free reward path remains open; the repaired parser admits 16,844 executed tool interactions. This isolates access to sampled tool experience from learning from that experience: the first changes by more than three orders of magnitude, while held-out rescues do not change. Practitioners should therefore verify both a **non-trivial tool-execution count in the first training step** and a held-out measure of the behaviour the tool is meant to teach. A rising reward curve establishes neither by itself.

6 Limitations

Seventeen limitations, in the order a sceptical reader should apply them, are stated in full in Appendix H; the six that most constrain what may be concluded are here.

The scale result is within one family. The $0 \rightarrow 4 \rightarrow 21 \rightarrow 36 \rightarrow 80$ ladder is Qwen2.5-Coder alone, against one mismatched interface, on one task (item 3). We claim a within-family monotone relation between checkpoint size and absolute undercount; we do not claim censoring is generally scale-increasing, and we do not equate parameter count with capability. A second *family*’s ladder is the test that would promote or refute it.

The repaired-FC control is one seed at one budget (item 8). It is a clean parser-registry intervention at 7B, but both arms use LoRA and 150 steps on one code task. It establishes the experience-distribution effect in this setting; its null learning result is not a theorem that a working channel can never help, nor is it strictly comparable to the historical 1.5B full-parameter runs.

Near-zero samples are not zero policy probability (items 10–11). The broken 7B arm retains full rollout text and distinguishes 8,689 tight-classified call emissions from 10 accepted and executed calls, so attempts and discards are directly separated there. The older 1.5B run did not retain full text and remains over-determined; neither run identifies the support of the policy beyond its sampled trajectories.

Conditioning on a successful parse is post-treatment conditioning (item 9). Whether an item parses is determined by the interface under test, so the both-parsed subset is not a random subsample. On Qwen the residual is +8.4 pp, 95% CI $[-0.5, +17.4]$, $p=0.118$: **we do not detect a residual difference, which is not the same as establishing there is none.**

Multiple comparisons (item 6). The explicit 27-test retrospective family in Appendix I uses $0.05/27 \approx 0.00185$. This reporting rule was not preregistered. Surviving it: the official-template `strict: true` intervention on wrong-tool labels ($p = 4.77 \times 10^{-7}$), its final-pass contrast ($p = 7.63 \times 10^{-5}$), and the protocol gap on 300 random items ($p=3.1 \times 10^{-6}$). **Not surviving it:** the Llama strict-to-ReAct contrast ($p = 0.0019$), the both-parsed Llama contrast ($p = 0.0021$), the repair-loop pass gain ($p=0.093$ at $n=100$, $p=0.0385$ at $n=300$), the Llama protocol contrast at $n=300$ ($p=0.0352$), the τ -bench outcome difference ($p=0.25$), and the residual-gap estimates ($p=0.118, 0.125$). We rest no claim on the latter group.

The historical training verifier had two exploitable seams (item 14): an unanchored `passed` regex and a denominator omitting skipped/expected-failure tests. We audited the historical

logs; before the formal 7B control we also anchored the regex, counted all pytest outcomes, and retained complete rollout text with hashes. Both P3 arms use that same repaired verifier, so verifier changes cannot explain their contrast; the older learning curves remain supported by an audit rather than a rerun.

Also constraining, and stated in full in the appendix: one task family and one tool (item 1); a non-random 100-item set, whose 7B rung the random 300-item replication moves from 21 to 30 (item 2); a deliberately asymmetric taxonomy in which Mistral has no admissible FC comparison at all (item 4); single-sample headline arms (item 5); scale stopping at 32B (item 7); sufficiency tested by one repaired-FC seed and budget (item 12); a stratified validation sample with a 4000-character output cap that creates false negatives while classifier imprecision creates false positives, leaving the raw count’s net bias unidentified (item 13); dependence on vLLM 0.27.1, verl 0.9.0 and publisher-mutable chat templates (items 15–16); and the instrumentation errata we disclose rather than absorb — seven arms recording `max_tokens` they did not run at, Llama’s initiation rate corrected from 97 to 74, and one previously reported $p=0.648$ withdrawn (item 17, Appendix A).

What would change our conclusions

We name these so the claims are falsifiable rather than merely hedged. *Replications or a longer repaired-FC budget.* The completed single-variable control restores 16,844 tool interactions without moving held-out rescues. A consistent rescue gain over additional seeds or a longer preregistered horizon would narrow the scope of this 150-step, single-seed observation; continued nulls would strengthen it within, but not beyond, the tested model/task setting. *A second family’s scale ladder,* with a working test executor: if the undercount does not grow with checkpoint size elsewhere, §4.3’s claim is a Qwen-plus-hermes fact rather than a scaling phenomenon. *The same ladder on a non-code task.* The five Qwen2.5-Coder checkpoints run against τ -bench **retail** under the documented configuration — models and interface fixed, only the domain varied. If the emitted column still rises with size while the parsed column stays at zero, the scale result is a property of the interface; if it does not, it is a property of code-shaped payloads. This needs a rescue arm at the small sizes for the reason Granite did: a checkpoint that simply never attempts a call produces an uninformative zero, not a censored one. *The same five-layer measurement on further benchmarks.* Done for BFCL v4 and τ -bench **retail**, so the concern is not specific to our harness and now covers one interactive suite; suites with much larger tool schemas, or whose scoring depends on argument-level matching, remain untested.

7 Conclusion

Tool-call interfaces censor agent trajectories, silently and per checkpoint; within Qwen2.5-Coder the absolute undercount rises monotonically with checkpoint size, which we report as a within-family observation rather than a general law. In a 150-step RL control differing only in verl’s parser registry, the mismatched arm records 8,689 tight-classified call-like emissions but only 10 executions, whereas the repaired arm executes and observes 16,844 calls. The intervention therefore restores the tool-mediated experience distribution. We detect no held-out improvement: rescues are 12 at both endpoints and final passes move from 430/542 to 427/542 under programmatic feedback. A post-hoc common native FC evaluation likewise detects no repaired-versus-broken success gain (248/542 each, with different passing sets). Access to a trajectory and learning from it are separate empirical questions.

Two controls fix what the effect is a property of. A 2×2 over chat template and parser on a public benchmark finds zero recovery from either one-component replacement at the documented

baseline and 196/200 parses under joint replacement; and a supporting ladder under a *matched* interface holds the silent fraction at 0–2 across a comparable span of scale, where the mismatched interface takes it to 80. **What the serving layer removes is not a property of the model, nor of the parser, but of the contract between them.**

The most uncomfortable finding is methodological. We ran three single-variable paired controls on a 23% failure and concluded it was a model deficiency; a fourth, taken from a sentence in the serving documentation, reduced it to zero. Our own human validation reproduced the same mechanism: twelve of ninety-eight annotations missed a tool call because it sat after a fenced code block, and the annotator — like the parser — concluded from the same bytes that no call had been made. Reporting server-parsed tool-call rates as model capability is not a safe default.

References

- Agentic rl training example: Keyerror: 'solve_equation'. verl issue #4124, 2025. <https://github.com/verl-project/verl/issues/4124>.
- Tool calling — vllm documentation, 2026. https://docs.vllm.ai/en/stable/features/tool_calling/.
- Arash Ahmadian, Chris Cremer, Matthias Gallé, Marzieh Fadaee, Julia Kreutzer, Olivier Pietquin, Ahmet Üstün, and Sara Hooker. Back to basics: Revisiting reinforce-style optimization for learning from human feedback in llms. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL), Volume 1: Long Papers*, pages 12248–12267, 2024.
- Wael Albayaydh, Rui Zhao, and Ivan Flechais. Beyond the leaderboard: A synthesis of tool-use, planning, and reasoning failures in large language model agents. *arXiv preprint arXiv:2607.05775*, 2026. Submitted 2026-07-07.
- Amazon AGI. τ^2 -bench-verified: a corrected and human-verified release of τ^2 -bench. GitHub repository, released 2025-11-26, last updated 2026-04-02, 2025. <https://github.com/amazon-agi/tau2-bench-verified>; defect categories and per-task justifications in its FIXES.md.
- Victor Barres, Honghua Dong, Soham Ray, Xujie Si, and Karthik Narasimhan. τ^2 -bench: Evaluating conversational agents in a dual-control environment. *arXiv preprint arXiv:2506.07982*, 2025. Submitted 2025-06-09; no conference venue found as of 2026-09-01.
- Vishvesh Bhat, Jay Vaghasiya, Muhammad Ahmed Mohsin, and Asad Aali. Benchmarking the benchmarks: A validity audit of tool-calling evaluation. *arXiv preprint arXiv:2607.02577*, 2026. Submitted 2026-06-30.
- Jingyi Chen, Songqiang Chen, Hengcheng Zhu, Jialun Cao, Jiasi Shen, and Shing-Chi Cheung. Understanding agent-reactive bugs at the model-harness boundary: An empirical study of llm agent issue reports. *arXiv preprint arXiv:2607.15684*, 2026.
- Mostafa Dehghani, Yi Tay, Alexey A. Gritsenko, Zhe Zhao, Neil Houlsby, Fernando Diaz, Donald Metzler, and Oriol Vinyals. The benchmark lottery. *arXiv preprint arXiv:2107.07002*, 2021.
- Chaitanya Vilas Garware and Sharif Noor Zisad. When the ruler is broken: Parsing-induced suppression in llm-based security log evaluation. *arXiv preprint arXiv:2605.07293*, 2026. Submitted 2026-05-08.

- hanXen. Feature request: dedicated tool parser for qwen2.5-coder models. vLLM issue #32926, opened 2026-01-23, closed as not planned, 2026. <https://github.com/vllm-project/vllm/issues/32926>.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *ACM SIGOPS Symposium on Operating Systems Principles (SOSP)*, 2023.
- Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. Api-bank: A comprehensive benchmark for tool-augmented llms. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.
- Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation. In *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 2023.
- LiveKit. Your model isn’t bad at tool calling. your serving stack is. LiveKit engineering blog, 2026-07-06, 2026. <https://livekit.com/blog/your-model-isnt-bad-at-tool-calling>.
- Moran Mizrahi, Guy Kaplan, Dan Malkin, Rotem Dror, Dafna Shahaf, and Gabriel Stanovsky. State of what art? a call for multi-prompt llm evaluation. *Transactions of the Association for Computational Linguistics (TACL)*, 12:933–949, 2024.
- Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. The berkeley function calling leaderboard (bfcl): From tool use to agentic evaluation of large language models. In *International Conference on Machine Learning (ICML)*, 2025. <https://gorilla.cs.berkeley.edu/leaderboard.html>.
- Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. Toolllm: Facilitating large language models to master 16000+ real-world apis. In *International Conference on Learning Representations (ICLR)*, 2024.
- Harsh Raj, Vipul Gupta, Anas Mahmoud, Razvan-Gabriel Dumitru, Darwin Yi, Aakash Sabharwal, and Yunzhong He. Model or harness? an interaction-centric taxonomy for localizing agent failures. *arXiv preprint arXiv:2607.28802*, 2026. Submitted 2026-07-30.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 2023.
- Melanie Sclar, Yejin Choi, Yulia Tsvetkov, and Alane Suhr. Quantifying language models’ sensitivity to spurious features in prompt design, or: How i learned to start worrying about prompt formatting. In *International Conference on Learning Representations (ICLR)*, 2024.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

- Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. Hybridflow: A flexible and efficient rlhf framework. In *Proceedings of the Twentieth European Conference on Computer Systems (EuroSys)*, 2025.
- Harsh Soni. Toolfailbench: Diagnosing tool-use failures in llm agents. *arXiv preprint arXiv:2607.04686*, 2026. Submitted 2026-07-06; workshops on Agents in the Wild (AIWILD) and Failure Modes of Agentic AI (FAGEN) at ICML 2026.
- Zhi Rui Tam, Cheng-Kuang Wu, Yi-Lin Tsai, Chieh-Yen Lin, Hung-yi Lee, and Yun-Nung Chen. Let me speak freely? a study on the impact of format restrictions on performance of large language models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track (EMNLP Industry)*, pages 1218–1236, 2024.
- Brandon T. Willard and Rémi Louf. Efficient guided generation for large language models. *arXiv preprint arXiv:2307.09702*, 2023.
- Zhangchen Xu, Yang Liu, Yueqin Yin, Mingyuan Zhou, and Radha Poovendran. Kodcode: A diverse, challenging, and verifiable synthetic dataset for coding. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 6980–7008, 2025. doi: 10.18653/v1/2025.findings-acl.365.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ -bench: A benchmark for tool-agent-user interaction in real-world domains. In *International Conference on Learning Representations (ICLR)*, 2025.
- Yilun Yao, Xinyu Tan, Chao-Hsuan Liu, Yaoming Li, Zhengyang Wang, Wenhan Yu, Zhewen Tan, Yuxuan Tian, Guangxiang Zhao, Lin Sun, Xiangzheng Zhang, and Tong Yang. Harness-bench: Measuring harness effects across models in realistic agent workflows. *arXiv preprint arXiv:2605.27922*, 2026.

A Data Errata

Every number in this paper traces to a trajectory file under `runs/final/`. This appendix lists the files whose recorded provenance is wrong, the arms that are inadmissible for pass-rate comparison, and what was verified correct. It is reproduced from `runs/final/ERRATA.md` in the released repository.

A.1 Seven ReAct arms record the wrong `max_tokens`

The following files record `max_tokens=2048` while the generation limit in force was **1024**. Cause: `gen()` in the probe carried a default of `max_tokens=1024` while `gen_fc()` used the global `MAX_TOKENS=2048`, and the provenance written to disk recorded `MAX_TOKENS` for both. Fixed 2026-08-31.

<code>traj_v3_Llama8B_react.jsonl</code>	recorded 2048 / actual 1024
<code>traj_v3_Qwen7B_react.jsonl</code>	recorded 2048 / actual 1024
<code>traj_v3_Mistral7B_react.jsonl</code>	recorded 2048 / actual 1024

<code>traj_v11_DS1.3b_react.jsonl</code>	recorded 2048 / actual 1024
<code>traj_v11_DS6.7b_react.jsonl</code>	recorded 2048 / actual 1024
<code>traj_v11_Llama32_1B_react.jsonl</code>	recorded 2048 / actual 1024
<code>traj_v11_Llama32_3B_react.jsonl</code>	recorded 2048 / actual 1024

Direction of the bias. The contemporaneous FC arms were genuinely at 2048, so in every affected ReAct-vs-FC comparison **the function-calling side had the larger generation budget**. A larger budget need not improve performance, so this asymmetry alone does not establish the direction of bias. Re-running the three main ReAct arms at a true 2048 changed final pass by at most one item ($80 \rightarrow 80$, $74 \rightarrow 74$, $32 \rightarrow 33$), which is why the correction is small in practice.

Replacement data. The four v11 arms are superseded by `traj_v13*_react.jsonl`; the three v3 arms by `traj_v14*_react.jsonl`. All tables in this paper cite the v13/v14 versions; the v3/v11 ReAct arms are retained only as a record.

A.2 Llama-3.1-8B initiation rate required manual correction

`traj_v3_Llama8B_fc.jsonl` records an initiation rate of 97/100, but 23 of those calls target **the task function rather than run_tests**. The probe did not verify tool names when that batch was collected, so `has_action` did not distinguish them. **The corrected run_tests initiation rate is 74/100.**

A re-run restricted to exactly those 23 items under matched configuration (`traj_v8_Llama8B_recheck.jsonl`, seed fixed) reproduced 23/23 as wrong-tool calls, with names such as `can_form_word`, `check_password_strength` and `floyd_warshall`.

Of the same batch, `traj_v3_Qwen7B_fc_nosuffix.jsonl` has an initiation rate of 0 and needs no correction; `traj_v3_Mistral7B_fc.jsonl` has 2, and those two were not individually re-checked for tool name.

A.3 Arms inadmissible for pass-rate comparison

<code>traj_v3_Mistral7B_fc.jsonl</code>	<code>n_err = 2</code>	<code>(rc=2)</code>
<code>traj_v5_Mistral7B_fc_official.jsonl</code>	<code>n_err = 42</code>	<code>(rc=2)</code>
<code>traj_v9_Mistral7B_fc_strict.jsonl</code>	<code>n_err = 3</code>	<code>(rc=2)</code>
<code>traj_v9_Mistral7B_fc_official_strict.jsonl</code>	<code>n_err = 39</code>	<code>(rc=2)</code>
<code>traj_v13_Llama8B_fcstrict_t06_s3.jsonl</code>	<code>n_err = 1</code>	<code>(rc=2)</code>

For these arms the **error-rate census remains valid** — that census is itself a result — but these arms are excluded from clean-arm pass-rate comparisons, and no clean-arm p -value is computed for them. Separately reported end-to-end analyses retain the full task denominator, including request failures. An earlier draft computed $p = 0.648$ on one such arm; that number is retracted.

A.4 Provenance schema drift

Fields were added over the course of the work, giving four schema generations: early arms lack `seed`, `temperature` or `fc_schema`. The fields load-bearing for every claim (`model`, `protocol`, `adapter`, `max_tokens`, `clean_index`) are present in all generations. A `script_sha256` field was never successfully added — the patch was interrupted twice — so script identity across a run is instead guaranteed externally, by hashing the probe before launch (`v13_pinned_hashes.txt`) and re-checking after.

A.5 Verified correct

- All 50 full-length arms use the identical 100-item set (`clean[:100]`; unique item-set count = 1), so every cross-arm comparison in this paper is exactly paired.
- Every FC arm ran at a true `max_tokens` of 2048.
- Sampling in the variance experiment is genuine: at `temperature` = 0.6, two seeds differ in first-turn code on 95/100 items and flip outcomes on 16/100.

A.6 Third-party component versions

The Qwen2.5-Coder `<tools>` parser is taken from `hanXen/vllm-qwen2.5-coder-tool-parser` (Apache 2.0), downloaded 2026-08-31, at the then-current `main`:

```
commit 1b921501f30cbfe347dccb1db7de3c82a1d55131
(1b92150, 2026-04-29)
"fix: buffer partial <tools> prefix in streaming
to prevent tag leak"

SHA256 (first 20):
c16bb1f88936a2d96c7c  qwen2_5_coder_tool_parser.py
                        (unmodified)
736bd175adbf90942c1d  tool_chat_template_qwen2_5_coder.jinja
                        (MODIFIED)
a95b2a9b91e65b3d452f  tool_chat_template_qwen2_5_coder.jinja.orig
                        (original)
```

The template was modified because its few-shot examples invoke `get_weather`, a tool absent from our schema. Left unchanged it would induce calls to a non-existent tool, simultaneously inflating the initiation count and contaminating the empty-argument statistic. The original is retained alongside.

B Reproducibility

The formal 7B configuration and shared held-out protocol are in Appendix D; the checkpoint curve and accepted resume lineage are in Appendix G.10. Appendix I defines the interval and retrospective test family. The anonymous package contains reduced Llama outcomes, 11,952 reduced 7B evaluation records, compact events including excluded segments, lineage metadata and verification scripts. It also includes a small raw-content mechanism archive: 800 first responses from the BFCL factorial, 100 documented and 100 repaired single-turn BFCL records with request schemas and official-scoring inputs, all 230 τ -bench task trajectories, and ten deterministically selected step-1 RL trajectories per arm. The full training rollout archive, original evaluation programs, training logs and heavy checkpoints are not included.

Verification target	Package support
Llama paired tests from reduced item records	Yes
Other historical tests from stated discordances	Yes, counts only
7B checkpoint outcomes and endpoint pairing	Yes
7B recorded funnel and accepted resume segments	Yes
Inspect/replay supplied BFCL raw outputs	Yes
Inspect τ -bench and sampled RL raw contents	Yes
Reclassify every training emission	No
Verify checkpoint weights or raw text hashes	No

From the extracted package root, run:

```
python3 analysis/verify_supplement.py
python3 analysis/submission_statistics.py --output /tmp/stats.json
python3 analysis/bfcl_fixed_replay.py \
    --bundle mechanism_evidence/bfcl
```

The event verifier uses the Python standard library and reconstructs inclusion from segment/step ranges. Dispatch, execution and observation can be uniquely joined. Extraction lacks per-turn request identity, so accepted-to-dispatch matching is only at trajectory-count granularity. Sampled RL events preserve raw emissions and recorded observations, but omit training task inputs and executable tests: they support text inspection, not independent environment re-execution. BFCL request fields come from its inference log, not a byte-for-byte HTTP request capture. Hashes do not substitute for file contents.

Full-archive-only procedures include `analysis/p3_final_evidence.py`, `analysis/intent.py`, `runs/final/final_table.py`, checkpoint audits and historical figure regeneration. They require data not supplied in the anonymous package and are not its default verification workflow. Historical trajectories and their configuration index reside under `runs/final/` in the full research archive; formal 7B records are retained separately. Only explicitly listed reductions and raw-content subsets are provided in the package.

Model snapshots and serving configurations must be pinned because template inheritance can change with revision. The third-party parser is `hanXen` (1b92150, Apache 2.0); paired few-shot examples were rewritten from `get_weather` to `run_tests`. Original and adapted hashes are retained in the full archive.

The post-hoc native FC evaluation adds `native_fc/outcomes.json` (1,626 reduced item records), `native_fc/statistics.json`, and `analysis/native_fc_results.py`. From the package root run `python3 analysis/native_fc_results.py --outcomes native_fc/outcomes.json --out /tmp/native-fc-recheck`. This reproduces paired contrasts and the timeout inventory; it does not re-execute generated programs. Full raw conversations and GPU loading receipts are retained separately, not promised as part of this reduction.

C Prompts and tool schemas, verbatim

All prompts are reproduced exactly as sent, in the original Chinese. We do not translate them: the protocol comparison is between two prompts, and a translation is a third prompt. An English gloss follows each block for readers who do not read Chinese, but the gloss was never sent to any model.

C.1 ReAct

Two strengths. `optional` permits a direct answer; `mandatory` requires an `Action` first. The main table uses `optional` on both protocols so that neither side is coerced.

REACT_OPTIONAL

你是一个 `Python` 编程助手，可以按以下格式工作：

Thought: 你的思考

Action: `run_tests`

Action Input: ``python

```

<完整代码>
...
Observation: (由系统填写测试结果)

Thought: 我已确认代码正确
Final Answer: ```python
<最终代码>
...

```

你可以用 `Action` 提交代码验证，也可以直接给 `Final Answer`。

Gloss: “You are a Python coding assistant and may work in the following format: Thought / Action: run_tests / Action Input: (fenced code) / Observation (filled in by the system) / ... / Final Answer: (fenced code). You may submit code with Action for verification, or give a Final Answer directly.”

The mandatory variant is identical except that the closing line becomes **必须先用 Action 提交代码验证，再给 Final Answer**。 (“You must first submit code for verification with Action, then give a Final Answer”), and the loop is marked as repeatable.

C.2 Function calling

FC_OPTIONAL

你是一个 Python 编程助手。请根据用户的描述实现所要求的函数。

你可以调用 `run_tests` 工具把代码交给测试运行，工具会返回通过情况与报错，你可以据此修改代码再次提交。

FC_MANDATORY

你是一个 Python 编程助手。请根据用户的描述实现所要求的函数。

****你必须先调用 `run_tests` 工具验证你的代码****，不要直接给出答案。
工具会返回测试通过情况与报错；若有失败，请修改代码并再次调用 `run_tests`。

Gloss: optional — “You may call the run_tests tool to submit code to the test runner; it returns pass/fail and errors, and you may revise and resubmit.” mandatory — **“You must first call the run_tests tool to verify your code; do not answer directly.”**

Strength is matched across every training comparison we report. The historical FC-ReAct comparison and both arms of the formal parser control use mandatory, so the near-closed broken channel cannot be attributed to a weaker instruction.

C.3 Tool schemas: terse, rich, strict

terse is the schema a reader arrives at by following the API documentation. It is the schema under which Llama’s 23% wrong-tool rate appears.

```

terse
-----
{"type": "function", "function": {
  "name": "run_tests",
  "description": "把你写的 Python 代码交给测试运行，返回通过情况与报错。",
  "parameters": {"type": "object",
    "properties": {"code": {"type": "string"}},
    "required": ["code"]}}}

```

Gloss: “Hand the Python code you wrote to the test runner; it returns pass/fail status and errors.”

`rich` is the §4.5 control. Its purpose is to test whether the wrong-tool rate is simply an under-specified schema, so it is written to be maximally explicit: the description states that the tool is *not* the function the user asked for, and the parameter description carries a worked example.

```
rich
----
{"type": "function", "function": {
  "name": "run_tests",
  "description": "把你写的 Python 代码提交给测试运行器执行，返回测试通过情况与报错。"
    "注意：这个工具不是你要实现的那个函数，不要把题目函数的参数传进来；"
    "唯一的参数 code 是你写的完整源代码文本。",
  "parameters": {"type": "object",
    "properties": {"code": {
      "type": "string",
      "description": "完整的 Python 源代码文本，包含所需的 import 和完整的"
        "函数定义。例如：\n\"def add(a, b):\n    return a + b\n\""},
    "required": ["code"]}}
```

Gloss of the added text: “Note: this tool is not the function you are being asked to implement. Do not pass the task function’s parameters into it. Its only parameter, `code`, is the complete source text you wrote.”

Under this explicit warning 22/100 items still pass the task function’s parameters, and they are the same parameter names that failed under `terse` (§4.5, $p=1.000$).

`strict` is `terse` plus `"strict": true` on the function object, with `VLLM_ENFORCE_STRICT_TOOL_CALLING=true` (the default). This is the single change that takes the wrong-tool rate to zero.

C.4 The Thought-scaffold control (§4.5)

To test whether ReAct’s advantage is merely an extra reasoning step, the FC system prompt was given a Thought requirement and nothing else changed:

“First write out your reasoning in a Thought: what the problem asks for, what the edge cases are, what algorithm you intend to use. Once you have thought it through, call the `run_tests` tool...”

The wrong-tool rate rose from 23 to 59 ($p < 0.001$). We report this as a single wording, not as a claim about reasoning scaffolds in general.

C.5 The role-disambiguation few-shot (§4.8)

Used at training scale (1.5B) after the wrong-tool failure was found to be semantic rather than syntactic. It shows the wrong call and the right call side by side.

你是一个 Python 编程助手。请根据用户的描述实现所要求的函数。

****你必须先调用 `run_tests` 工具验证你的代码****，不要直接给出答案。

注意一个常见错误：****用户描述里要你实现的那个函数，不是可调用的工具。****
你唯一可以调用的工具是 ``run_tests``，它只有一个参数 ``code``，内容是你写的完整源代码。

错误示范（用户要求实现 `join_integers`）：

```
{"name": "join_integers", "arguments": {"int_array": [1, 2, 3]}}
```

—— 这是在调用你自己该编写的函数，`run_tests` 收不到任何代码。

正确示范：

```
{"name": "run_tests", "arguments": {"code": "def join_integers(int_array):\n    return ','.join(map(str, int_array))"}}}
```

—— 函数名恒为 `run_tests`，你写的整段代码作为字符串放进 `code`。

工具会返回测试通过情况与报错；若有失败，请修改代码并再次调用 `run_tests`。

Gloss: “You are a Python coding assistant. Implement the function the user describes. **You must first call the `run_tests` tool to verify your code**; do not answer directly. Note a common mistake: **the function the user asks you to implement is not a callable tool**. The only tool you may call is `run_tests`; it has a single parameter `code`, holding the complete source text you wrote. *Wrong* (the user asked for `join_integers`): `{"name": "join_integers", "arguments": {"int_array": [1, 2, 3]}}` — this calls the function you were supposed to write, and `run_tests` receives no code at all. *Right*: `{"name": "run_tests", "arguments": {"code": "def join_integers(...)"}}` — the function name is always `run_tests`, and your whole program goes into `code` as a string. The tool returns pass/fail and errors; if any fail, revise and call `run_tests` again.”

It made things worse: recoverable calls rose $52 \rightarrow 64$ while calls naming `run_tests` stayed at **0** and final pass fell $15 \rightarrow 13$. The model became more fluent at emitting the wrong call.

C.6 The intent criterion

Emission indicators are benchmark-specific and held fixed across arms within each benchmark. For KodCode, the regex below targets `run_tests` and a quoted code payload with Python-like lexical markers. It rejects some schema echoes and placeholders but does not establish JSON validity, Python executability, or semantic intent. Strong and weak use disjoint trajectory-label sets; tight is computed independently.

For BFCL, the JSON-call indicator detects an embedded decodable JSON object naming a tool offered in the request and containing object-valued arguments, including arguments encoded as a JSON string. Format-marker and JSON-marker indicators are independent broader checks; they overlap and must not be summed. Counts refer to unparsed first-request responses, with at most one count per response for each indicator.

For τ -bench, the unparsed call-shaped indicator detects a generic named-call pattern or a Hermes tool-call marker in assistant messages without structured tool calls. It does not validate membership in the offered tool set or guarantee syntactic validity. KodCode annotation precision is not assumed to transfer to either external benchmark.

```
TIGHT = re.compile(
    r'"name"\s*:\s*"run_tests"\.{0,200}? "arguments"\s*:\s*\{'
    r'\.{0,80}? "code"\s*:\s*(\.{0,4000}?)"\s*\}', re.S)

REAL = re.compile(r'\n|def |class |return |import |lambda ')

def is_tight_call(raw):
    m = TIGHT.search(raw or "")
    return bool(m and REAL.search(m.group(1)))
```

The regex requires **name** before **arguments** and a closing argument brace after the quoted code field; valid alternative key orders may be missed. KodCode human validation is in §3.1. Inter-annotator $\kappa = 0.967$ is restricted to items presented as identical bytes.

D Serving and training configuration

D.1 Per-family serving flags

Baseline provenance and scope. The vLLM 0.27.1 tool-calling documentation, under “Qwen Models”, lists Qwen/Qwen2.5-* with the hermes parser (https://docs.vllm.ai/en/v0.27.1/features/tool_calling/#qwen-models; checked 23 September 2026). Our Coder baseline applies this family-level guidance; the passage does not separately validate a Coder checkpoint. Thus “documented” denotes the origin of the configuration choice, not a Coder-specific compatibility guarantee or evidence of its prevalence in deployment. Issue #32926 already described this mismatch before our experiments. Our contribution is its controlled measurement and downstream consequences, not discovery of the incompatibility or invention of the community repair.

These are the configurations under test, including documented serving flags and the adapter/schema interventions identified in our experiments. The rightmost column summarizes observed limitations or failure symptoms; it is not a uniform flag-omission ablation.

Table 4: Serving configurations and observed symptoms (vLLM 0.27.1).

Family	Flags / intervention	Observed symptom
all	<code>--enable-auto-tool-choice</code>	HTTP 400 if omitted
Qwen2.5 (In-struct)	<code>--tool-call-parser hermes</code>	1–88/100 parsed across sizes; unparsed outputs include payload failures
Qwen2.5-Coder	Baseline: <code>hermes</code> ; repair: community parser plus paired template	baseline 0 parsed across sizes; joint adapter 196/200 on BFCL
Llama 3.1 / 3.2	<code>--tool-call-parser llama3_json</code> <code>--chat-template tool_chat_template_llama3.1_json.jinja</code> plus <code>"strict": true</code> in the schema	malformed arguments; without <code>strict</code> the terse baseline has 23% wrong-tool calls
Mistral (HF format)	<code>--tokenizer-mode hf --config-format hf</code> <code>--load-format hf</code> <code>--tool-call-parser mistral</code> <code>--chat-template tool_chat_template_mistral_parallel.jinja</code> tool_call_id must be exactly 9 characters	HTTP 400; the officially recommended parallel template <i>raises</i> the error rate from 2% to 42%
DeepSeek-Coder	none available: the chat template never injects tools	n/a

Missing auto-tool-choice support and the tested Mistral configurations produce explicit request errors. Other mismatches can return HTTP 200 with empty `tool_calls`, indistinguishable at that field from declining to call a tool. The released 98-line `preflight_toolcall.py` checks a canonical request for a non-empty call with the right name and parseable arguments, then repeats under `tool_choice: required` as a positive control. This is a screening check, not a guarantee of compatibility on all tasks.

D.2 Training configuration

The historical 1.5B runs use the following configuration; they motivate the investigation but are not the formal parser-repair control:

Table 5: Training configuration. All arms share every value below; arms differ only in the protocol and, for the historical runs, the algorithm and seed.

Setting	Value
Model	Qwen2.5-Coder-1.5B-Instruct
Framework	verl 0.9.0 (vLLM 0.27.1 rollout backend)
Algorithm	GRPO ($\times 2$ seeds) and RLOO ($\times 1$ seed)
Trajectories per step	<code>train_batch_size</code> $16 \times \text{rollout.n}$ $8 = 128$
Steps	150 for the historical runs and formal 7B control; 10 / 3 / 75 for the historical diagnostic arms in Appendix G
Training data	KodCode, decontaminated pool of 7669 items
Held-out evaluation	Multi: 164 HumanEval+, 378 MBPP+; repair: 155 HumanEval+, 299 MBPP+
Denominators	Formal: 542 multi + 454 repair records; historical multi: 540 after excluding two defective items
Agent loop	<code>tool_agent</code> (FC) or <code>react_agent</code> (ReAct)
Tool	<code>run_tests</code> , single <code>code</code> argument, sandboxed <code>pytest</code>
System prompt	FC_MANDATORY / ReAct mandatory — matched strength (Appendix C)
Hardware	single A800 80 GB

The formal control of §4.8 instead uses Qwen2.5-Coder-7B-Instruct, GRPO, LoRA rank and alpha 32 over all linear modules, seed 0, 150 steps, and the same $16 \text{ prompts} \times 8 \text{ rollouts}$ per step in both arms. Both use the mandatory FC prompt and fixed verifier, retain complete rollout text, and evaluate the same 542 paired multi-turn items at steps 0/30/60/90/120/150. Evaluation uses a shared parser-independent programmatic-feedback protocol: the current LoRA is queried through verl’s native rollout RPC at temperature 0, for at most four turns and 1,024 tokens per turn; generated code is executed against EvalPlus tests and the observation is appended as a user message. Native RPC metadata must match the expected global weight step. The sole arm difference is `multi_turn.format=hermes` versus the registered `qwen2_5_coder` parser. Full-parameter 7B RL did not fit the single 80 GB card, so both arms use the same parameter-efficient configuration. Logged training settings are learning rate 10^{-6} , temperature 1.0, top- $p = 1$, top- $k = -1$, prompt budget 2,048 and response budget 6,144 tokens, at most five assistant turns, a 1,200-token tool-response cap and one dispatched tool call per generation. These are training budgets, distinct from the four-turn, 1,024-token-per-turn held-out evaluation. Decontamination uses question n-grams 8/5 and solution n-grams 10/6, with a five-gram minimum; containment is intersection divided by the smaller set, and the clean-ID selection threshold is 0.10. These checks do not rule out semantic overlap or pretraining contamination. This preserves the arm contrast but prevents a strict comparison to the historical full-parameter curves.

D.3 Tool-call instrumentation

verl 0.9.0’s step metrics contain **no** tool-call count. The metric named `timing_s/agent_loop/tool_calls` is *elapsed seconds*; reading it as a count is a hard error and we flag it because the name invites exactly that.

We therefore instrumented the two agent-side call sites directly, each writing one labelled line per call:

- `CodeTool.execute` — the function-calling path
- `ReActAgentLoop._run_tests` — the ReAct path

The counter must not be placed in `sandbox.run_tests`. The reward function evaluates the final program through that same function, so a counter there conflates reward evaluation with agent tool use. We hit this and moved the counter; the sandbox’s own counter was renamed `sandbox_exec_count` so the two can never be confused again.

In the historical diagnostic run, the FC counter file has **0 lines**; the ReAct arm’s has **788**, all carrying the `react` label, so the provenance is uncontaminated. Three mutually independent signals agree — `num_turns` pinned at its minimum of 2, tool time exactly 0.00s, and a dedicated counter at 0.

D.4 Constructing the repaired-FC training arm

The control initially appeared blocked at three layers:

1. `multi_turn.format=hermes` selects **verl’s own** `ToolParser` registry, which is independent of vLLM’s — so installing a vLLM parser plugin has no effect on the training path.
2. A verl-side parser we wrote does recover calls, but **none of them are the tool**: 36/52 target the task function, 16/52 carry executable code elsewhere in the body. Name normalisation cannot repair this, because no call carries code to normalise.
3. The 1.5B checkpoint emits no correctly named call for that parser to accept, and 7B full-parameter training does not fit a single 80 GB card.

The first two observations determine the intervention; the third determines the scale and LoRA configuration. We registered the parser inside verl and ran the matched 7B pair. Thus the earlier statement that no executable repaired-FC arm existed was a description of the configuration surface, not a permanent limitation; §4.8 reports the completed control.

D.5 The path to the repaired-FC training arm

We encountered four successive obstacles while constructing the control. Each attempt is a measurement, so we retain the record; all were eventually resolved by registering the parser in verl and moving to matched 7B LoRA arms.

(i) **A vLLM parser plugin does not reach the training loop.** verl’s `AgentLoop` performs its own tool-call parsing (`verl/experimental/agent_loop/tool_parser.py`, registered as `hermes`, adapted from vLLM v0.9.1). `multi_turn.format=hermes` selects *that* parser, not the server’s, so attaching a plugin to the vLLM rollout server changes nothing in training — a distinction worth stating because the two look identical from the outside.

(ii) **A verl-side parser recovers calls, but none of them are the tool.** We implemented a `qwen2_5_coder` parser inside verl’s registry accepting `<tools>` tags, `json` fenced blocks and bare JSON. In the historical 1.5B diagnostic with a recorded mandatory-prompt file override ($n=100$), it lifts recoverable calls from **0/100 (hermes)** to **52/100**. The original override file is unavailable locally, so exact prompt-byte equivalence to training cannot be verified (Appendix E). Of those 52, **zero name run_tests** (Table 27). Every call targets the function the task asks the model to *write* — `can_form_word({"tiles", "word"})`, `check_password_strength({"password"})` — the same signature as Llama’s failure in §4.5, now at a $5\times$ smaller model. **Name normalisation cannot repair this: no call carries code to normalise.**

(iii) **A role-disambiguation few-shot makes it worse, not better.** Since the failure is semantic rather than syntactic, we targeted it directly: a system prompt containing the wrong call and the right call side by side, with an explicit warning that the task function is not a tool. Same model, same 100 items, same server; only the system prompt changes. The intervention made the

model *more fluent at emitting the wrong call* (52 $\rightarrow$ 64 recoverable) and slightly worse at the task (15 $\rightarrow$ 13), with the count that matters staying at zero (Table 28). This is the third independent instance of the pattern in §G.9.

(iv) **Constrained decoding is unavailable in this stack.** Forcing the tool choice, or schema-constrained decoding, is not exposed by verl 0.9.0’s vLLM rollout path (its only `strict` key belongs to the profiler). This is a limitation of the tested rollout path, not an argument against constrained decoding. Parser repair accepts existing emissions, whereas generation constraints alter the output distribution. The latter is a distinct potential remedy for serialization or allowed-name violations; neither remedy by itself establishes correct arguments or successful task behavior.

E Arm inventory

Every full-length arm, generated from the trajectory files’ own provenance rather than transcribed by hand. All 50 arms share one item set (`clean[:100]`; distinct-set count verified = 1), so every cross-arm comparison in the paper is exactly paired. Fields recorded as --- were absent from that generation’s provenance schema (Appendix A.4); this is disclosed rather than back-filled. Seven arms carry a known-wrong `max_tokens` record and are listed in Appendix A.1; five are inadmissible for pass rates and are listed in Appendix A.3.

The `strength` column is the recorded CLI selector, not necessarily the effective prompt. For `a14` and `roleprobe`, records specify `sys_file`; the archived probe loads this file after selecting the built-in prompt, overriding `strength=optional`. The retained role-disambiguation file explicitly mandates `run_tests`. The original `sys_fc_mandatory.txt` file was not found in the local archive, so the `a14` run’s exact prompt bytes cannot be independently verified; its name alone is not proof. The zero correctly named calls remain an observed outcome, not a fully byte-matched prompt replication.

Not listed: `v8_Llama8B_recheck` (23 items, a targeted qualitative re-run) and eight `*smoke*` files at `n=3`. Neither is a formal arm.

Table 6: All 50 full-length arms and their recorded configuration.

Arm	Model	Prot.	Strength	Schema	Adapter	tok	temp	seed
<code>a14_Qwen1.5B_fc_mandatory</code>	Qwen-1.5B	fc	optional	terse	<code>cross_family</code>	2048	—	—
<code>roleprobe_Qwen1.5B_fc_rolendisambig</code>	Qwen-1.5B	fc	optional	terse	<code>cross_family</code>	2048	0.0	0
<code>v11_DS1.3b_react</code>	DS-1.3B	react	optional	terse	<code>cross_family</code>	2048	—	—
<code>v11_DS6.7b_react</code>	DS-6.7B	react	optional	terse	<code>cross_family</code>	2048	0.0	—
<code>v11_Llama32_1B_fc_strict</code>	Llama-3.2-1B	fc	optional	strict	<code>cross_family</code>	2048	0.0	—
<code>v11_Llama32_1B_react</code>	Llama-3.2-1B	react	optional	terse	<code>cross_family</code>	2048	0.0	—
<code>v11_Llama32_3B_fc_strict</code>	Llama-3.2-3B	fc	optional	strict	<code>cross_family</code>	2048	0.0	—
<code>v11_Llama32_3B_react</code>	Llama-3.2-3B	react	optional	terse	<code>cross_family</code>	2048	0.0	—
<code>v13_DS1.3b_react</code>	DS-1.3B	react	optional	terse	<code>cross_family</code>	2048	0.0	—
<code>v13_DS6.7b_react</code>	DS-6.7B	react	optional	terse	<code>cross_family</code>	2048	0.0	—
<code>v13_Llama32_1B_react</code>	Llama-3.2-1B	react	optional	terse	<code>cross_family</code>	2048	0.0	—
<code>v13_Llama32_3B_react</code>	Llama-3.2-3B	react	optional	terse	<code>cross_family</code>	2048	0.0	—
<code>v13_Llama8B_fcstrict_t06_s1</code>	Llama-3.1-8B-Instruct	fc	optional	strict	<code>cross_family</code>	2048	0.6	1
<code>v13_Llama8B_fcstrict_t06_s2</code>	Llama-3.1-8B-Instruct	fc	optional	strict	<code>cross_family</code>	2048	0.6	2
<code>v13_Llama8B_fcstrict_t06_s3</code>	Llama-3.1-8B-Instruct	fc	optional	strict	<code>cross_family</code>	2048	0.6	3
<code>v13_Llama8B_react_t06_s1</code>	Llama-3.1-8B-Instruct	react	optional	terse	<code>cross_family</code>	2048	0.6	1
<code>v13_Llama8B_react_t06_s2</code>	Llama-3.1-8B-Instruct	react	optional	terse	<code>cross_family</code>	2048	0.6	2
<code>v13_Llama8B_react_t06_s3</code>	Llama-3.1-8B-Instruct	react	optional	terse	<code>cross_family</code>	2048	0.6	3
<code>v14_Llama8B_react</code>	Llama-3.1-8B-Instruct	react	optional	terse	<code>cross_family</code>	2048	0.0	—
<code>v14_Mistral7B_react</code>	Mistral-7B	react	optional	terse	<code>cross_family</code>	2048	0.0	—
<code>v14_Qwen7B_react</code>	Qwen-7B	react	optional	terse	<code>cross_family</code>	2048	0.0	—
<code>v3_Llama8B_fc</code>	Llama-3.1-8B-Instruct	fc	optional	—	<code>cross_family</code>	2048	—	—
<code>v3_Llama8B_fc_legacy</code>	Llama-3.1-8B-Instruct	fc	optional	—	<code>cross_family</code>	2048	—	—
<code>v3_Llama8B_react</code>	Llama-3.1-8B-Instruct	react	optional	—	<code>cross_family</code>	2048	—	—

Arm	Model	Prot.	Strength	Schema	Adapter	tok	temp	seed
v3_Mistral7B_fc	Mistral-7B	fc	optional	terse	cross_family	2048	—	—
v3_Mistral7B_react	Mistral-7B	react	optional	terse	cross_family	2048	—	—
v3_Qwen7B_fc_legacy	Qwen-7B	fc	optional	terse	cross_family	2048	—	—
v3_Qwen7B_fc_nosuffix	Qwen-7B	fc	optional	terse	cross_family	2048	—	—
v3_Qwen7B_react	Qwen-7B	react	optional	terse	cross_family	2048	—	—
v4_Llama8B_fc_cot	Llama-3.1-8B-Instruct	fc	optional	terse	cross_family	2048	—	—
v4_Llama8B_fc_rich	Llama-3.1-8B-Instruct	fc	optional	rich	cross_family	2048	—	—
v5_Llama8B_fc_official	Llama-3.1-8B-Instruct	fc	optional	terse	cross_family	2048	—	—
v5_Mistral7B_fc_official	Mistral-7B	fc	optional	terse	cross_family	2048	—	—
v5_Qwen1.5B_fc_intent	Qwen-1.5B	fc	optional	terse	cross_family	2048	—	—
v5_Qwen14B_fc_intent	Qwen-14B	fc	optional	terse	cross_family	2048	—	—
v5_Qwen32B_fc_intent	Qwen-32B	fc	optional	terse	cross_family	2048	—	—
v5_Qwen3B_fc_intent	Qwen-3B	fc	optional	terse	cross_family	2048	—	—
v5_Qwen7B_fc_intent	Qwen-7B	fc	optional	terse	cross_family	2048	—	—
v6_Llama8B_fc_strict	Llama-3.1-8B-Instruct	fc	optional	strict	cross_family	2048	—	—
v6b_Qwen1.5B_fc_plugin	Qwen-1.5B	fc	optional	terse	cross_family	2048	—	—
v6b_Qwen3B_fc_plugin	Qwen-3B	fc	optional	terse	cross_family	2048	—	—
v6b_Qwen7B_fc_plugin	Qwen-7B	fc	optional	terse	cross_family	2048	—	—
v9_Mistral7B_fc_official_strict	Mistral-7B	fc	optional	strict	cross_family	2048	—	—
v9_Mistral7B_fc_strict	Mistral-7B	fc	optional	strict	cross_family	2048	—	—
x2_DS1.3B_react	DS-1.3B	react	optional	—	—	—	—	—
x2_DS6.7B_react	DS-6.7B	react	optional	—	—	—	—	—
x2_Llama8B_fc	Llama-3.1-8B-Instruct	fc	optional	—	—	—	—	—
x2_Llama8B_react	Llama-3.1-8B-Instruct	react	optional	—	—	—	—	—
x2_Mistral7B_fc	Mistral-7B	fc	optional	—	—	—	—	—
x2_Mistral7B_react	Mistral-7B	react	optional	—	—	—	—	—

F Human validation of the intent criterion, in full

The scale trend in §4.3 rests on an automated classifier, so we validated it against human judgement. 98 outputs were sampled **stratified by classifier verdict** (40 **tight**, 28 **strong-but-not-tight**, 30 **neither**) — deliberately over-sampling the decision boundary so both false positives and false negatives are estimable. Sampling is seeded and the annotator saw only the raw output, never the classifier’s label. We report **two rounds**, because the first is itself a finding.

Table 7: Human validation of the intent criterion, round 1 and its adjudication. The classifier is used only to count what the server discarded, so its recall matters more than its precision.

	agreement	Cohen’s κ	precision	recall	FP	FN
Round 1 (raw output only)	86.7%	0.713	70.0%	96.6%	12	1
After adjudication	96.9%	0.936	95.0%	97.4%	2	1

The 12 round-1 false positives split into two causes, and the second is ours, not the annotator’s. **Eleven are reading-order failures:** the tool call sits mid-document, after a fenced ````python` block (match positions 1209–2585 in outputs of median length ~ 2000); the annotator read the code block, concluded “this is a direct answer, not a call”, and stopped. **One is an instrument failure:** for a 32B item the matched span begins at character 2585 and the annotation pack truncated every output at 2400, so the evidence was *not in the pack at all* and that item could not have been labelled correctly. Re-shown the matched span alone, 10 of 13 disputed items were reversed. The corrected pack carries every output in full.

The first adjudication was one-sided, and we do not treat its κ as a reliability estimate. Auditing the procedure after the fact: the 13 adjudicated items were selected *exactly* as the items where the annotator disagreed with the classifier (13/13 overlap), and all 10 reversals moved *toward* the classifier, none away. Under that selection rule agreement can only rise, so

0.713 $\rightarrow$ 0.936 is a property of which items were re-examined, not evidence of reliability. We report the second annotator instead.

A second independent annotator

A second annotator, blind to the classifier, to the first annotator’s labels and to the first pack’s item order, labelled the identical 98 items under a verbatim-identical rubric. Two changes were made to the instrument: outputs are carried **in full**, and the adjudication rule was fixed in advance to review *all* three-way disagreements rather than only those disagreeing with the classifier.

Table 8: Inter-annotator reliability. Of 98 sampled items, 97 have complete paired A1/A2 records. The identical-byte subset removes a presentation confound introduced by the first pack’s 2,400-character cap.

Cohen’s κ (95% CI, bootstrap)	all items ($n=97$)	identical bytes ($n=62$)
A1 vs. A2 (inter-annotator)	0.736 [0.595, 0.866]	0.967 [0.893, 1.000]
A2 vs. classifier	0.936 [0.852, 1.000]	0.967 [0.896, 1.000]
A1 vs. classifier	0.712 [0.564, 0.849]	0.934 [0.829, 1.000]

Of the 12 paired disagreements, 11 have full text exceeding 2,400 characters and A1 saw a different presentation from A2; the first-round audit attributed most errors to reading order and one to evidence actually omitted by truncation. We therefore do not assign all 11 to one cause. Repeated A1–A2 intervals use 5,000 bootstrap resamples, seed 7 and sorted item IDs; earlier differing bounds reflected inconsistent bootstrap inputs/order, not different annotators. On the 62 items where both read identical bytes the two agree on 61, and **inter-annotator $\kappa = 0.967$** .

Third-party adjudication, and the final labels. Fifteen items were not unanimous across A1, A2 and the classifier. All fifteen went to a third adjudicator under the pre-registered rule — selected by *any* three-way disagreement, never by disagreement with the classifier — who saw full text and no one else’s labels. The adjudicator sided with A2 on 11, with the classifier on 9, and with A1 on 6, and **moved away from the classifier on 6 of 15**. That last number is the check that matters: the first round’s adjudication moved away from the classifier zero times out of thirteen, which is what an outcome looks like when the selection rule has already decided it. Combining the 83 unanimous items with the 15 adjudicated ones gives a 98-item gold standard. The third adjudicator was blind to classifier and annotator labels; the classifier nevertheless affected the original stratified sample and which non-unanimous items required adjudication, so we do not call the gold set classifier-independent.

Table 9: Validation against the adjudicated gold standard. Pairwise denominators differ because one sampled item lacks a complete A1/A2 pair; classifier–gold uses all 98 items.

Cohen’s κ (95% CI, bootstrap)	full n	full sample	identical bytes ($n=62$)
A1 vs. A2 (inter-annotator)	97	0.736 [0.595, 0.866]	0.967 [0.893, 1.000]
Classifier vs. gold standard	98	0.871 [0.756, 0.958]	0.934 [0.829, 1.000]

Sensitivity estimate, not a corrected population total. Tight-stratum precision is 36/40=0.900. If this precision transfers to the 32B arm, multiplying 80 by 0.900 gives approximately 72 classifier-positive outputs expected to satisfy the gold criterion. This does not restore false negatives and is not an estimate of all true calls. The stratified gold sample contains 4 false positives and 2 false negatives; their unweighted difference is not a population bias estimate.

Uniformly applying 0.900 yields 0, 3.6, 18.9, 32.4, 72 by construction, but scale-specific precision is unmeasured, so this calculation does not independently validate the trend.

For the record, this number moved twice as the validation improved, and we report the trajectory rather than only its endpoint: **0.950** under the first, one-sided adjudication; **0.975** under A2’s labels alone; **0.900** against the adjudicated gold standard. The first was inflated by its selection rule, the second rested on a single rater, and only the third is built from a rule fixed before the labels were seen.

A bound on the measurement itself. The probe stores at most 4000 characters of raw output per item; 10 of the 98 sampled outputs reach that ceiling, and one grey-zone item is a call truncated by it. The classifier and both annotators read identical bytes, which is what agreement requires, but a call emitted beyond character 4000 is invisible to all three. All ten cap hits occur in the 32B subset of this stratified sample: by increasing scale the hit counts are 0/6, 0/13, 0/15, 0/24, and 10/40. These are sample diagnostics, not population truncation rates, but they show that the uncertainty is concentrated at the headline rung. The cap therefore creates downward pressure through false negatives. Because the classifier also has false positives (tight precision 0.900), however, the net bias of the raw count is not known; we report both the uncorrected count and the assumption-dependent sensitivity estimate rather than calling the total a strict lower bound.

G Supplementary results

G.1 Historical observations that motivated the study

Historical 1.5B runs showed numerical pass gains of 2.6–2.8 points ($p = 0.064/0.078/0.077$), not statistically significant. Of newly passing items, 91–94% passed on turn 1; multi-turn rescues ranged from 5 to 10 out of 540 across the reported checkpoint sequences. These small exploratory counts prompted the investigation but do not identify an interface effect or independently establish absence of learning.

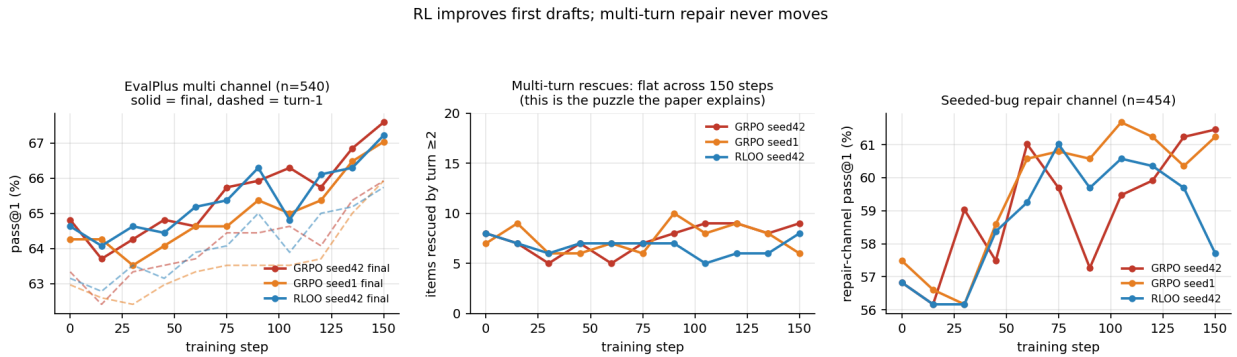


Figure 2: Left: pass@1 rises over 150 steps on all three runs, driven by first drafts (dashed) rather than final answers. Centre: multi-turn rescues range from 5–10 out of 540 across checkpoints, algorithms and seeds. Right: the seeded-bug repair channel does improve, confirming the model is learning something about fixing code; no clear increase in multi-turn rescue counts was detected in these historical runs. §4.8 shows the tool was never successfully called in any of these runs.

G.2 Result tables referenced from §4

Table 10: BFCL v4, first HTTP request per case. Emission indicators are measured only where parsing failed; JSON-call, format-marker and JSON-marker checks overlap and must not be summed.

Arm	cases	HTTP err	first parsed	unparsed	JSON-call	format	JSON
hermes (documented)	200	0	0	200	166	169	187
qwen2_5_coder (repaired)	200	0	196	4	0	1	2

Table 11: τ -bench **retail**, all 115 tasks paired across arms (none dropped).

	documented	repaired
assistant turns	1389	1971
server-parsed calls	0	636
emitted but unparsed	789	6
tool executions / observations returned	0	636
tasks entering the tool loop at all	0	103
tasks solved	7	10

Table 12: Same probe, items and criterion under a *matched* envelope.

Size	server-parsed	silent count / 100 (tight, unparsed)
Qwen3-0.6B	86	1
Qwen3-1.7B	44	1
Qwen3-4B	70	2
Qwen3-8B	97	0
Qwen2.5-Coder 1.5B–32B (mismatched)	0 at every size	0 $\rightarrow$ 4 $\rightarrow$ 21 $\rightarrow$ 36 $\rightarrow$ 80

Table 13: Qwen2.5-Instruct under the identical mismatched configuration, with both ladders’ task outcomes measured on a working executor.

Size	Instruct: parsed / unparsed indicators			Coder		Instruct		
	parsed	tight	weak	parsed	turn-1 = final	turn-1	final	rescued
1.5B	1	0	7	0	31	13	13	0
3B	11	56	2	0	37	14	17	+3
7B	63	21	0	0	53	34	40	+6
14B	77	19	0	0	53	38	52	+14
32B	88	8	0	0	68	42	65	+23

Table 14: Llama-3.1-8B arms, paired on the same 100 items.

Arm	Calls issued	Wrong-tool	Executed	Turn-1	Final
terse (parser only)	97	23	74	34	49
+ official template	95	22	73	33	44
+ official template + strict	98	0	97	46	61
ReAct (reference)	98	0	97	61	80

Table 15: Decomposing the ReAct-FC gap on Llama, paired McNemar.

Comparison	b/c	p	Descriptive difference
terse-FC $\rightarrow$ strict-FC (49 $\rightarrow$ 61)	15/3	0.0075	interface repair, +12
strict-FC $\rightarrow$ ReAct (61 $\rightarrow$ 80)	27/8	0.0019	protocol, +19
terse-FC $\rightarrow$ ReAct (49 $\rightarrow$ 80)	38/7	3.1e-06	both, +31

Table 16: Repair loop on Qwen2.5-Coder-7B. Tested steps count generations with extracted code sent to the sandbox, not all assistant outputs. The historical ReAct row uses v3; the unified v14 rerun has mean tested steps 1.93 with unchanged pass/rescue counts.

Arm	Parsed	Mean tested steps	≥ 2 tested steps	Turn-1	Final	Rescued
ReAct	95	1.92	40	57	74	17
FC + hermes	0	0.96	0	53	53	0
FC + dedicated adapter	84	1.82	37	53	62	9

Table 17: RL training rollouts, matched in model, data, hyper-parameters, seed and prompt strength.

Arm	Steps	num_turns/mean	Tool time	Tool calls / rollout
FC (tool_agent + hermes)	10	2.000 (constant)	0.00 s	0.000
ReAct (react_agent)	3	5.883	12.21 s	2.052

G.3 Offline re-parse matrix and failure-layer decomposition

To exclude “your extractor is simply better than **hermes**”, we re-parsed the *same stored bytes* under four rules (`analysis/reparse_matrix.py`, no GPU).

Table 18: The same bytes, re-parsed offline under four rules. Where the server already parsed a call, **content** is empty and offline replay has nothing to read; those cells are N/A rather than zero. The gap is a parser mismatch, not a claim that no parser could have read the output.

Arm (server-parsed = 0)	server	hermes replay	<tools> replay	bare JSON	tight
Qwen-1.5B	0	0	0	0	0
Qwen-3B	0	0	0	5	4
Qwen-7B	0	0	0	22	21
Qwen-14B	0	0	0	42	36
Qwen-32B	0	0	0	100	80

The **hermes** column is zero at every scale: the model never produces that format. The bare-JSON and tight columns are not: the calls exist, and their difference from the server column is precisely what the format mismatch removes. For arms where the server *did* parse, offline content-only re-parsing is **undefined, not zero** — a successful parse moves the call into `tool_calls` and leaves `content` empty.

Replaying vLLM 0.27.1’s **hermes** extractor line for line over the stored bytes (`analysis/failure_layer.py`) separates the layer at which each item is lost: no `<tool_call>` envelope at all; envelope present but the JSON payload malformed; payload well-formed and rejected only because `json.loads` does not tolerate trailing text in the capture; or a genuine parser loss, meaning the replay succeeds where the server did not.

Table 19: Where the unparsed items are lost. These are first-failure categories of all unparsed outputs, not counts of identified tool attempts. **Genuine parser loss is zero in every arm** — and across all 4254 de-duplicated first turns in the archive.

Arm	no envelope	payload malformed	strictness only	parser loss
Coder 1.5B–32B (each size)	100	0	0	0
Instruct 1.5B	93	6	0	0
Instruct 3B	22	41	26	0
Instruct 7B	3	33	1	0
Instruct 14B	13	6	4	0
Instruct 32B	1	9	2	0

Missing envelopes alone do not identify censoring: the 100 Coder outputs at each size include both call-shaped and non-call outputs (notably at 1.5B). A separate emission criterion is needed to identify candidate masking. Instruct outputs reaching the payload check may break JSON, almost always by embedding Python whose docstring quotes or raw newlines are left unescaped inside the code string. And **in no arm — across the whole archive — did the server fail to parse a call that its own extractor would have accepted**. “Censoring” throughout this paper therefore means *the attempt never reaches the parser in a form it is defined to accept*, not *the parser discards valid calls*.

The repair-loop table uses the stored `n_turns` counter, which advances only when code is extracted and tested. The Hermes arm has 100 first assistant outputs but four yield no code, hence 96/100 tested steps. Its mean assistant-output count is 1.00; the adapter’s is 1.90, rather than its 1.82 tested-step mean. Reaching a second tested step is stricter than merely receiving feedback: a subsequent unextractable response does not increment this counter.

G.4 BFCL: gating and the full reproduction record

Fixed-output replay on single-turn cases. We added a post-hoc, answer-blind replay of all 100 `simple_python` documented-arm responses from the clean run. A deterministic extractor scans left to right for maximal JSON objects or arrays, accepts named calls with object arguments (including JSON-encoded argument strings), and retains all recovered calls. It neither repairs malformed JSON nor selects a call using the answer key. All 97 recovered responses name an offered tool and pass structural signature checks (required fields, types, declared parameter names, array items and enums). Format annotations are not validated. Of the 100 original responses, 92 pass the official BFCL AST checker at the experiment’s pinned commit. The 100 repaired-arm outputs reproduce 96 official successes as a positive control. Scoring code is unchanged; the offline loader supplies the same `underscore_to_dot=True` registry property without importing online inference SDKs.

This is an additional offline interpretation rule, not the community dedicated parser: none of the original 100 responses contains its required `<tools>` start tag. The result demonstrates that correct single-turn calls can be recovered from fixed emitted bytes. It does not attribute the remaining difference to template changes alone, establish a safe general-purpose fallback parser, or recover a counterfactual multi-turn success rate. Changing a multi-turn action would change subsequent environment feedback and generations. The supplement includes raw responses, the recorded request schemas, answer keys, per-item verdicts and the pinned checker sources; see Appendix B.

Each arm passes a two-case end-to-end smoke test before its full run, and a run is rejected unless the result count, the score file’s `total_count` and the number of distinct case ids in the recording proxy all agree at 200 with zero HTTP errors. Both arms run at `max_model_len = 16384`,

`temperature = 0` and inference seed 0, on cases drawn at manifest seed 20260901. Registration of BFCL’s OpenAI handler happens in memory at import time, so `bfcl_eval`’s own files are left byte-identical; a recording proxy sits between benchmark and server and logs every request.

We ran the pair twice. The **hermes** arm is identical across runs in every column: 0/200 parsed, 166 tight, 480 requests, 0 errors. The repaired arm moves by one case between runs — **197/200 versus 196/200** first parsed, **99/100 versus 98/100** executed — while success is 19 in both. Greedy decoding does not make vLLM bitwise deterministic across differing batch compositions, and this arm runs eight threads through a multi-turn executor, so we report its intermediate funnel columns as ± 1 rather than exact.

Table 20: BFCL `multi_turn_base`, $n = 100$, joined to the HTTP trace by case id.

Arm	tight (unparsed)	first parsed	any parsed	executed	success	rescued
hermes	69	0	0	0	0	0
qwen2_5_coder	0	97	98	98	19	0

Table 21: Two quantified failure layers; the full four-family table is Table 22.

Family	Layer	Symptom	Detectable in advance?	Remedy
Qwen2.5-Coder	parser	emits a fenced JSON block, not <code><tool_call></code>	$\times$ template check false-positives	dedicated adapter
Llama-3.1-8B	schema	calls the <i>task function</i> as a tool (23%)	$\times$ requires argument inspection	strict: true

G.5 The full four-family taxonomy

Table 22: Four serving diagnostics, not an exhaustive taxonomy. Only Mistral’s failure announces itself.

Family	Layer	Symptom	Detectable in advance?	Remedy
DeepSeek-Coder	template	chat template never injects tools	$\checkmark$ template inspection	none
Qwen2.5-Coder	parser	emits a fenced JSON block, not <code><tool_call></code>	$\times$ template check false-positives	dedicated adapter
Llama-3.1-8B	schema	calls the <i>task function</i> as a tool (23%)	$\times$ requires argument inspection	strict: true
Mistral-7B-v0.3	token	repeats <code>[TOOL_CALLS]</code> $\rightarrow$ HTTP 400	$\sim$ errors, but only sometimes	none found

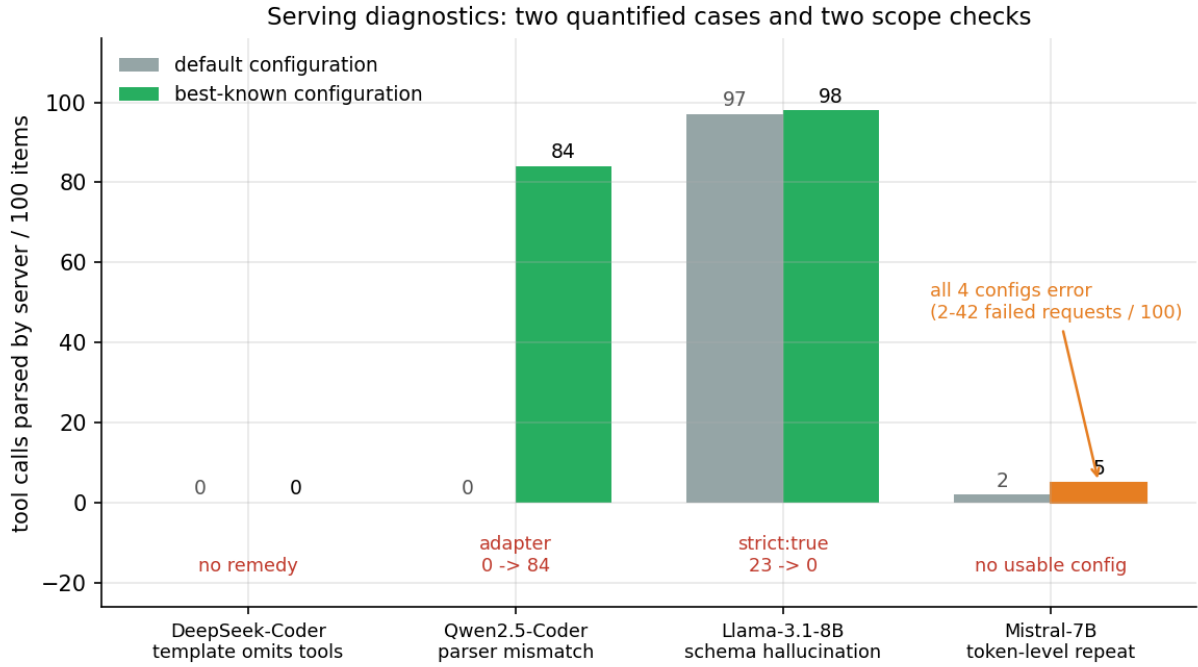


Figure 3: Serving diagnostics with a consistent server-parsed metric. Llama is 97 versus 98 parsed; wrong-tool and executed counts are reported separately. Grey is the configuration a reader arrives at by following the documentation; green is the best configuration we found. Mistral never produces an error-free run under any of four configurations, so its bars are not comparable with the others and are drawn in orange.

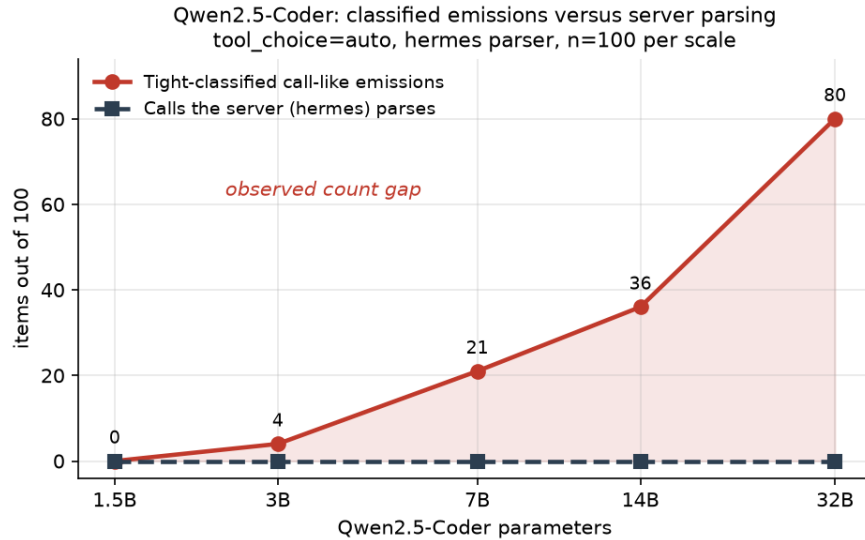


Figure 4: Within-family observation under the mismatched interface. Across the five Qwen2.5-Coder checkpoints, the server parses zero calls while tight-classified emissions rise to 80/100 at 32B (approximately 72 as a sensitivity estimate using tight-stratum precision 0.900, assuming transfer across sizes; Appendix F). Same data as Table 2.

G.6 Main table under unified configuration

Table 23: All arms genuinely at `max_tokens=2048`, one shared 100-item set, `temperature=0`. Mistral has no admissible FC arm: all four configurations produce request errors, so no pass rate and no p -value are computed.

Model	ReAct		FC		b/c	p	FC arm
	turn-1	final	turn-1	final			
Llama-3.1-8B	61	80	46	61	27/8	0.0019	official tmpl. + strict
Qwen2.5-Coder-7B	57	74	53	62	17/5	0.0169	dedicated adapter
Mistral-7B-v0.3	26	33	N/A	N/A	N/A	N/A	—

G.7 The four Llama controls, tabulated

Table 24: Four single-variable controls on Llama-3.1-8B’s 23% wrong-tool rate. The first three did not reduce the error; the fourth eliminated it through a schema constraint, without identifying a unique cause of role confusion.

Alternative explanation	Control	Wrong-tool calls / 100	Verdict
schema lacks a parameter description	rich vs terse	23 $\rightarrow$ 22 ($p=1.000$)	rejected
ReAct merely adds a reasoning step	FC + Thought scaffold	23 $\rightarrow$ 59 ($p<0.001$, worse)	rejected
server not configured per documentation	+ official chat template	23 $\rightarrow$ 22 (1/0 discordance, $p=1.000$)	rejected
schema constraint not enabled	+ strict: true on official template	22 $\rightarrow$ 0 (22/0, $p=4.77\times 10^{-7}$)	accepted

G.8 Conditioning on a successful parse, and access vs. use of feedback

Table 25: Conditioning on items where both protocols parsed successfully. These are descriptive post-treatment subsets; neither nominal test clears the 27-test Bonferroni threshold.

Family	Items both parsed	ReAct	FC	b/c	p
Qwen2.5-Coder-7B	83	63	56	11/4	0.118 (n.s.)
Llama-3.1-8B	96	78	60	25/7	0.0021

Table 26: Descriptive repair success among items reaching a second code-bearing, tested step. The subset is selected after treatment; these rates do not localize or causally decompose the protocol gap.

Arm	Reached tested step ≥ 2	Rescued	Repair success given feedback
ReAct	40	17	42.5%
FC + adapter	37	9	24.3%
FC + hermes	0	0	undefined

Adapters are not a universal switch. The same adapter recovers 84/100 at 7B and **1/100 at 3B**: the 3B checkpoint ignores the `<tools>` few-shot entirely (zero `<tools>` tags in 100 items, 99 direct code) while still solving the task at a comparable rate (final 53). Adapter effectiveness is

non-monotonic in scale, reinforcing the per-checkpoint claim of §4.6. The dedicated solution also changes parser, chat template and few-shot simultaneously; we additionally rewrote the template’s few-shot examples, which originally invoked `get_weather` — a tool absent from our schema, which would have induced calls to a non-existent tool and inflated both initiation and empty-argument counts. Original and modified hashes are recorded.

G.9 Pressure on a broken interface makes things worse

Two independent controls point the same way. Adding a reasoning scaffold to FC raised role confusion from 23 to **59** (§4.5). Replacing the optional tool instruction with a mandatory one on Qwen2.5-Coder-1.5B moved parser-accepted calls not at all — still 0/100 — while final pass fell from **31 to 15** and unparsable outputs rose from ~ 0 to 46/100. Neither intervention touches the interface, and both make the observable outcome worse. **When an agent appears reluctant to use tools, strengthening the instruction is the cheapest thing to try and the most likely to mislead** — it can degrade task performance while leaving the tool-call count at zero, which looks like a model that is both unwilling *and* incapable.

Table 27: Probe replication on the 1.5B model actually being trained, recorded mandatory-prompt override (n=100; exact original file unavailable, Appendix E). Not one output names `run_tests`, so at this scale the missing calls are not a parser mismatch — there was nothing well-formed to censor.

Category	n	%
wrong name, <code>arguments</code> carry complete code	0	0%
wrong name, <code>arguments</code> are the task function’s parameters	36	69%
wrong name, executable code present elsewhere in the body	16	31%

Table 28: Increasing pressure on the 1.5B model. Stronger instruction and few-shot examples raise the number of recoverable code payloads but never produce a call that names the tool.

	recoverable calls	naming <code>run_tests</code>	final pass
mandatory prompt (baseline)	52	0	15
+ role-disambiguation few-shot	64	0	13

G.10 Formal 7B checkpoint curve and final-lineage audit

All checkpoint evaluations use the same 542-item multi-turn manifest, have zero request failures, and attest the indicated native weight step. The accepted curve uses the original repaired steps 0–90 and resumed evaluation checkpoints 120 and 150; the abandoned original steps 91–116 are excluded. The repeated step-90 evaluation after resumption gave 415/426/11 rather than 415/428/13 and is retained as test–retest evidence, not substituted into the formal curve.

Table 29: Complete formal 7B held-out curve. Each cell is turn-1 pass / final pass / rescued after turn 1, with $n = 542$.

step	broken FC	repaired FC
0	418 / 430 / 12	418 / 430 / 12
30	418 / 430 / 12	418 / 429 / 11
60	420 / 433 / 13	417 / 431 / 14
90	418 / 432 / 14	415 / 428 / 13
120	419 / 433 / 14	415 / 427 / 12
150	418 / 430 / 12	415 / 427 / 12

The policies did update. The accepted logs contain all 150 steps with non-zero actor gradient norms: 0.0120–0.0251 for broken and 0.00641–0.01599 for repaired; mean training rewards span 0.150–0.469 and 0.168–0.435, respectively. A direct checkpoint audit over all 392 LoRA tensors (80,740,352 parameters) finds 45,471,572 values changed from broken step 120 to 150 (56.3%, ℓ_2 delta 0.0787) and 45,296,056 from repaired step 90 to resumed step 150 (56.1%, ℓ_2 delta 0.1222). These comparisons establish updates to saved LoRA weights over the retained intervals; evaluation completion records separately attest the native weight step.

The event join also resolves the funnel’s apparent gaps. Broken FC has 19,203 generation events, 8,689 tight emissions and 10 accepted/executed/observed calls. Repaired FC has 33,367 generation events, 17,608 tight emissions, and 16,912 accepted calls in 16,844 accepted generation events. Forty generations contain multiple calls, producing exactly the 68-call surplus; the configured `max_parallel_calls=1` selects the first returned call and leaves subsequent calls undispached (neither merged nor queued by this loop). This accounts for the 68-call difference. All 16,844 dispatches join to executions and observations by request ID and assistant turn. Extraction records lack those identifiers; accepted-generation and dispatch counts match per trajectory, rather than through a per-generation join. Tight and accepted are different tests: 16,323 events satisfy both, 1,285 are tight but unaccepted, and 521 are accepted but not tight. With the separately retained full archive (not included in the anonymous package), `analysis/p3_final_evidence.py` verifies all evaluation and event hashes, reconstructs the final lineage, and emits the per-file manifest; the server-side checkpoint audit is reproducible with `analysis/p3_checkpoint_lora_audit.py`.

G.11 Historical ReAct positive control, tabulated

Table 30: Historical ReAct training with a working tool channel, 23,676 executed calls. Rescues by turn ≥ 2 do not move. The three historical 1.5B FC runs, in which **zero** tool calls executed, give rescue sequences [7, 9, 6, 6, 7, 6, 10, 8, 9, 8, 6], [8, 7, 6, 7, 7, 7, 5, 6, 6, 8] and [7, 5, 7, 5, 7, 8, 9, 9, 8, 9]. This is supporting evidence only; Table 3 is the single-variable control.

step	turn-1	final	rescued by turn ≥ 2	repair channel
0	359	367	8	263
15	340	346	6	266
30	340	346	6	262
45	341	349	8	261
60	341	347	6	274

G.12 Sampling variance, and an inverted noise structure

Llama-3.1-8B, `temperature=0.6`, $n=100 \times 3$ seeds.

RL training rollouts: under FC the tool is never successfully called
Qwen2.5-Coder-1.5B, verl GRPO, mandatory tool-use prompt on both arms

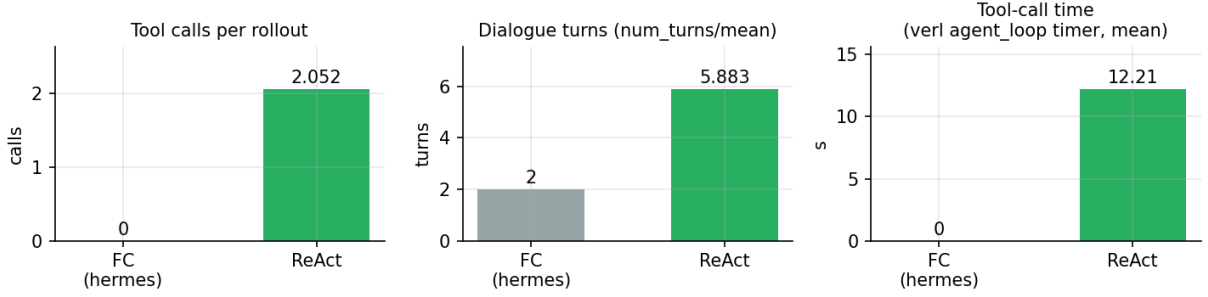


Figure 5: **Historical 1.5B RL diagnostic.** Under function calling the tool is never successfully called; `num_turns` stays pinned at its minimum of 2 and tool time is exactly zero. Prompt strength is matched across arms (both mandatory); step counts are not (10 vs 3). The formal 7B parser control is reported separately in Table 3.

Table 31: Sampling variance. FC seed 3 contains one rejected parallel-call request. We report its complete-denominator endpoint, and treat the two clean arms separately; an SD from two clean seeds would be too unstable to summarize variation reliably.

	seed 1	seed 2	seed 3	admissible
ReAct turn-1	53	56	43	3/3
ReAct final	72	72	72	3/3
FC turn-1	44	44	45	—
FC final	62	66	57 [†]	2 clean / 3 end-to-end

FC seed 3 contains one request error — the model emitted parallel tool calls and vLLM returned “*This model only supports single tool-calls at once*”. The end-to-end endpoint therefore retains that task as a failure and is 57/100; the prespecified clean-arm analysis excludes the seed and reports [62, 66]. An SD can be computed from two clean seeds, but would be an unstable estimate, so we do not use it to characterize sampling variation. Worst case among clean arms: ReAct’s lowest (72) versus FC’s highest (66) = +6.

Inverted noise structure. ReAct’s turn-1 varies widely (43–56, sd 6.8) while its final converges; FC’s turn-1 is nearly constant (44–45, sd 0.58) while its final disperses. The mechanism is compensation in the rescue count — 19 / 16 / **29**, largest where turn-1 was worst — and in all three seeds the count of “passed at turn 1 but failed finally” is **0**: the repair loop is monotone. **The three identical finals are a coincidence of totals, not identical item sets**, and sd = 0.00 must not be read as “ReAct is perfectly stable”.

G.13 Exploratory observations of stable totals and changing item sets

Two independent observations show the same structure. *Across sampling seeds*: ReAct’s final pass is 72 / 72 / 72 at `temperature=0.6`, yet the solved sets have pairwise Jaccard 0.71–0.80 — 15–24 items differ. *Across training steps*: between steps 15 and 30 of the ReAct run, 85 of 542 first-turn completions changed verbatim and 6 items flipped outcome (3 gained, 3 lost), leaving turn-1 and final pass numerically identical (340 / 346 at both checkpoints). The policy is demonstrably

moving; the aggregate is not.

These exploratory observations show that equal aggregate scores can conceal different item-level outcomes. Compensation through rescues is consistent with the observed counts, but these few runs do not identify a general stabilising effect of repair or establish a deployment benefit. We therefore report item-level changes alongside totals; both observations come from one model family and one task.

G.14 The training verifier: seams, audit, and planned fixes

The GRPO reward is outcome-only: it extracts the last submitted program, runs the item’s own `pytest` suite in the sandbox, and returns the pass ratio (`reward_code.py`). Three reward-hacking routes are closed by construction — a run in which no test executes scores 0 rather than 1, a timeout scores 0, and an item with no test string scores 0. The historical verifier nevertheless had two seams. (i) Its summary parser searched `(\d+) passed` over merged `stdout/stderr` and took the first match, so model `stdout` could imitate `pytest`’s summary. (ii) `skipped` and `xfail` were not counted in the denominator. Both were reachable in principle. We therefore audited: across the five training logs that retain generated code there are **zero** occurrences of a printed ```N passed```, of `pytest.skip/allow_module_level`, or of `sys.exit/os._exit`, and `critic/rewards/mean` oscillates in 0.46–0.73 without the step change a discovered exploit produces. The audit’s own limit is that historical full rollout text was not retained, so it covers only the fragments those logs carry. Before the formal 7B control we anchored the regex to `pytest`’s summary, included `skipped/xfail/xpassed` in the denominator, and retained every rollout in full with a SHA-256 digest. Both P3 arms use this fixed verifier. Neither verifier path is used for the reported held-out pass rates, which require `mode="script"`, a zero exit code and `status=="ok"`.

G.15 Post-hoc common native function-calling evaluation

This additional evaluation uses the existing base and final-lineage step-150 broken/repaired checkpoints, not new training. It was designed after review, outside the training preregistration. All three weights see the same 542 frozen EvalPlus items, original tokenizer/chat template and `run_tests` schema, and the same working repaired verl extraction logic. The BFCL few-shot adapter is not used. Each loaded LoRA has 392 tensors and 80,740,352 parameters; ordered float32 tensor digests match the audited checkpoint exports, with rank/alpha 32 and scaling 1. Base/tokenizer configuration hashes are checked; full base-weight shards were not independently rehashed. Generation uses sequential Transformers with bf16/SDPA, greedy decoding, seed 0, at most four assistant generations of 1,024 new tokens, an 8,192-token context budget and a 1,200-character observation cap. Context overflow fails explicitly, without silent truncation. Package versions and source hashes are retained in the configuration records.

Only an accepted, correctly named call with nonempty code triggers interactive execution. The first accepted call per generation is dispatched; additional calls would be ignored, but none occur here. Feedback is a tool-role message and full history is re-rendered on every turn. Passing a tool test alone does not stop the loop; a no-call response or the generation cap ends it. A later explicit final code submission supersedes the last tool code; plain closing prose does not. After the dialogue, final code is scored once independently with no returned feedback. That terminal test is not a tool invocation. Tests use EvalPlus script mode with a 30-second timeout, not the training reward’s `pytest` partial credit. This aligns tool-role feedback, but is not byte-identical replay of AgentLoop or an elimination of task/test distribution shift.

Here a rescue requires a failed first *actual tool test* followed by a passing tool test. An untested

Table 32: Common native FC evaluation, 542 paired tasks per weight. Calls are interactive tool executions; timeouts are terminal scoring timeouts.

Weight	Final pass	Initiating tasks	Calls	Rescues	Timeouts
Base	239	46	91	2	4
Broken step 150	248	50	93	1	5
Repaired step 150	248	47	90	1	6

first response is not a failure rescued later. Thus these counts are not interchangeable with the earlier automatic-feedback rescue metric. Accepted calls, executions and returned observations agree (91/93/90); generation counts are 619/622/619. No output hits the generation length cap. Every interactive execution completes with status `ok`; this denotes successful test execution, not necessarily code correctness. Terminal timeouts remain failures in the full 542-item denominator, with no clean-arm filtering. Base timeouts are MBPP 84, 126, 388 and 739; broken additionally times out on 603; repaired additionally on 603 and 765. These observed timeouts are not independently diagnosed as intrinsic nontermination. No missing submissions or other terminal infrastructure statuses are recorded.

For repaired minus broken final success, gains/losses are 16/16, difference zero percentage points, conservative paired 95% CI $[-3.47, 3.47]$, exact two-sided McNemar $p = 1$. Relative to base, broken has 20 gains and 11 losses (+1.66 points, CI $[-1.74, 5.03]$, $p = 0.150$), and repaired 22 gains and 13 losses (+1.66 points, CI $[-1.94, 5.23]$, $p = 0.175$). These intervals use the construction in Appendix I; they cover item-pair uncertainty, not training-seed or inference-retest variability. Equal totals do not imply equal passing sets or equivalent policies. Neither protocol detects a success benefit of repair in this setting; this does not establish that interface repair cannot improve learning.

The three weight contrasts crossed with final success, initiation and rescue form nine explicitly post-hoc exploratory tests, with Bonferroni-adjusted values supplied alongside nominal values. They are separate from the earlier 27-test reporting audit; none is significant even before adjustment. A single 36-test correction would not make any of the new results significant. The intervals are pointwise, not simultaneous. The deterministic six- and 24-task engineering pilots overlap the full set and are not counted as additional observations or seeds.

The package supplies 1,626 item records and `native_fc_results.py`. Running this CPU script with `--outcomes` recomputes every listed contrast and timeout inventory without GPU access. The reduced records do not contain generated code or full conversations; independent reclassification or execution requires the separately retained raw archive. Its `--archive` mode checks raw hashes, matching inputs, loaded adapter identity and execution-to-observation links before reduction.

H Limitations, in full

In the order a sceptical reader should apply them.

Scope of the evidence. (1) *One task family for the scale and training results.* Our own probe uses decontaminated KodCode items with a single tool taking one `code` argument. The external-validity results are *not* code-only: BFCL’s `multi_turn_base` spans file-system, vehicle and trading APIs, and τ -bench `retail` is a customer-service domain whose 789 unparsed turns decompose to 100% envelope-layer with zero parser loss, reproducing the code result (§4.2). What remains code-only is the *scale ladder* and the *RL contamination* result. The former is a real gap — the same ladder on a non-code task would test whether the undercount’s growth with checkpoint

size is a property of the interface or of code-shaped payloads (§6). The latter we do not expect to close: the reward is a `pytest` pass ratio, so porting it changes environment, reward and task at once. The controlled scale and RL experiments use a single-tool schema and do not isolate tool selection among alternatives. A single-tool schema is an easy parser case, but we treat that only as motivation for broader tests, not as a quantitative lower-bound argument. (2) *Item sampling*. The 50 full-length arms use 100 items taken as `clean[:100]`, **not a random sample**; pairing across arms is exact, so within-study comparisons are sound. Generalisation rests on a replication on a random 300-item sample (seed 20260901): server-parsed remains 0 at every size across 1500 items, and strict emitted counts scale to 0, 4, 30, 40, 81 per 100 against the original 0, 4, 21, 36, 80. **7B moves from 21 to 30**, so the original subset understated that rung; direction and monotonicity are unaffected, but a single 100-item subset is evidently not a stable estimate of an intermediate rung. (3) *The scale result is within one family*. The $0 \rightarrow 4 \rightarrow 21 \rightarrow 36 \rightarrow 80$ ladder is Qwen2.5-Coder alone, against one mismatched interface, on one task; we do not claim censoring is generally scale-increasing, and we do not equate parameter count with capability. The Instruct ladder is a control on the *mechanism*, not a second ladder of undercount — its execution layer was initially void (the host lacked `pytest`) and has since been re-run with a working executor, the parse layer reproducing value for value. A second *family*’s ladder remains the test that would promote or refute the scale claim. (4) *The taxonomy is four instances, not an enumeration*. The evidence is deliberately asymmetric: Qwen has a scale ladder, a replay matrix and a repair; Llama a single-variable intervention; **Mistral has no admissible 100-item FC comparison at all**; DeepSeek is a template check with little quantification. (5) *Single-sample headline arms*. Table 16 and the ladders are one sample at `temperature=0`. Variance is estimated only on Llama-3.1-8B, and there only 2 of 3 FC arms are clean. A standard deviation from two seeds is defined but too unstable to summarize sampling variation reliably; the complete-denominator endpoint for the error-bearing seed is still reported (§G.12). (6) *Multiple comparisons*. The explicit 27-test retrospective family in Appendix I uses $0.05/27 \approx 0.00185$. This reporting rule was not preregistered. Surviving it: the official-template `strict: true` intervention on wrong-tool labels ($p = 4.77 \times 10^{-7}$) and final pass ($p = 7.63 \times 10^{-5}$), the protocol gap on 300 random items ($p = 3.1 \times 10^{-6}$), and the official-template strict turn-1 contrast ($p = 0.000977$). **Not surviving it**: the Llama strict-to-ReAct contrast ($p = 0.0019$), the both-parsed Llama contrast ($p = 0.0021$), the repair-loop pass gain ($p = 0.093$ at $n=100$, $p = 0.0385$ at $n=300$), the Llama protocol contrast at $n=300$ ($p = 0.0352$), and the residual-gap estimates ($p = 0.118, 0.125$). We rest no claim on the latter group, and report the $n=300$ values precisely so a reader can see the direction is stable where significance is not claimed. (7) *Model scale stops at 32B*, on a single A800; no frontier models.

Threats to the causal reading. (8) *The clean repaired-FC control has one seed and one budget*. The completed 7B experiment holds model, LoRA configuration, data, prompt, decoding, verifier, seed and 150-step budget fixed, changing only the parser registered in `verl`. It therefore identifies the interface effect on the sampled experience distribution in this setting. Its null held-out result does not establish that a working channel is generally insufficient, and its LoRA training is not strictly comparable to the historical 1.5B full-parameter runs. (9) *Conditioning on a successful parse is post-treatment conditioning*. Whether an item parses is determined by the interface under test, so the both-parsed subset is not a random subsample and its contrast is not a clean causal estimate. On Qwen the residual is +8.4 pp, 95% CI $[-0.5, +17.4]$, $p = 0.118$: **we do not detect a residual difference, which is not the same as establishing there is none**. (10) *Near-zero observed frequency is not zero probability*. The broken 7B arm executes 10 calls rather than none; its instrumentation describes a sampled distribution, not the support of the policy. We therefore report a nearly closed channel (10 executions alongside 8,689 tight-classified generation events), not a mathematically unreachable action. (11) *Attempts and discards are separated only in the new*

run. The formal 7B arms retain complete rollout text with hashes, parser verdicts, execution events and returned observations, so 8,689 regex-positive emissions and 10 accepted actions are separately recorded; the classifier is not a semantic-validity oracle and its link to dispatch is a trajectory-level count check. The historical 1.5B run did not retain full text. Its same-prompt replication finds no correctly named call, but that cannot retroactively identify what every historical rollout emitted. (12) *The null learning observation is setting-specific.* Parser repair restores 16,844 executed and observed calls over 150 steps with 12 held-out rescues at both endpoints (intermediate checkpoints vary). This is the requested single-variable control, but it is one seed, 7B, LoRA, one code task and one training horizon. The earlier ReAct arm (23,676 calls, flat rescues) is directionally consistent but remains confounded by its protocol change.

The shared evaluation deliberately avoids both training parsers, but changes structured tool-role feedback to user-message feedback. It measures transfer to this programmatic-feedback protocol, not native FC behavior. Formatting shift, limited adaptation budget/capacity and multi-turn credit assignment are plausible explanations for absent transfer; these data do not separate them. The evaluator extracts the last decodable `<tool_call>` code argument when present, otherwise the last code fence or the raw text. This is not a general decoder for every tool syntax. The post-hoc common native FC evaluation (Appendix G.15) additionally uses voluntary calls and tool-role feedback: repaired and broken both pass 248/542, with 16 discordances each way. This addresses the missing initiation measurement, not all distribution shift: held-out tasks use script-mode tests, the controller re-renders history, and only one training seed is available. It is not a byte-identical training rollout or an equivalence test. The new rescue definition requires an actual failed tool test and must not be pooled with programmatic-feedback rescues.

(13) *Human validation covers one criterion on one sample.* The 98-item stratified sample has 97 complete paired annotator records; these give inter-annotator $\kappa = 0.967$ on the 62 items where both read identical bytes, and classifier-vs-gold-standard $\kappa = 0.871$ after third-party adjudication (Appendix F). Two residual limits: the sample is stratified by classifier verdict, so the unweighted sample metrics are not population precision or recall; and stored raw output is capped at 4000 characters, so a call emitted past that point is invisible to classifier and annotators alike, creating false negatives. Because the classifier also has false positives, the net bias of the raw emitted-call count is not identified. The *first* round’s apparent unreliability ($\kappa=0.713$) was confounded by 2400-character truncation in the annotation pack and by selective adjudication; the identical-byte analysis removes the presentation confound but does not establish that every disagreement was an instrument error. (14) *The historical training verifier had two exploitable seams.* Its summary parser used the first unanchored `(\d+)` `passed` match and omitted `skipped/xfail` from the denominator. We found no exploit signature in the five historical logs that retain generated code, but those logs do not contain every full rollout. For the formal 7B control we anchored the regex to `pytest`’s summary, included `skipped`, `xfailed` and `xpassed` outcomes in the denominator, and retained every rollout in full with a hash. Both arms use the identical fixed verifier, so it cannot explain their contrast. Neither verifier version is used for the separately reported held-out pass rates (§G.14).

Environment dependence and provenance. (15) *Serving-stack version dependence.* Every measurement is against vLLM 0.27.1 and verl 0.9.0, so parser behaviour, template defaults and the semantics of `strict: true` are properties of those versions. We pin versions, parser and template hashes, and model revisions for exactly this reason, and we have not replicated on a second serving stack (e.g. SGLang), so we cannot say which findings are vLLM-specific. (16) *Repository-revision dependence.* Chat templates ship inside `tokenizer_config.json` and can be changed by the publisher without a model change; the Qwen2.5-Coder false positive is precisely an artefact of an inherited template. (17) *Instrumentation errata, disclosed rather than absorbed.* Seven ReAct arms recorded `max_tokens=2048` while running at 1024 (an asymmetry that favoured FC; re-running

changed final pass by ≤ 1 item); Llama’s initiation rate required manual correction from 97 to 74 after tool-name validation was added; five arms are error-bearing and admissible only for the error-rate census, and one previously reported $p=0.648$ computed on such an arm is withdrawn. All are enumerated in Appendix A. None is silently absorbed into a reported number.

What would change our conclusions (restated)

We name these so the claims are falsifiable rather than merely hedged. *Replications or a longer repaired-FC budget.* The requested broken-versus-repaired single-variable control is now complete: parser repair restores 16,844 tool interactions and does not increase held-out rescues. A consistent rescue gain over additional seeds or a longer preregistered horizon would narrow the scope of this 150-step, single-seed observation; continued nulls would strengthen it within, but not beyond, the tested model, task and optimiser setting. *A second family’s scale ladder,* with a working test executor: if the undercount does not grow with checkpoint size in another family, §4.3’s scale claim is a Qwen-plus-hermes fact rather than a scaling phenomenon. *The same ladder on a non-code task.* The five Qwen2.5-Coder checkpoints run against τ -bench **retail** under the documented configuration — models and interface fixed, only the domain varied. If the emitted column still rises with size while the parsed column stays at zero, the scale result is a property of the interface; if it does not, it is a property of code-shaped payloads. This needs a rescue arm at the small sizes for the reason Granite did: a checkpoint that simply never attempts a call produces an uninformative zero, not a censored one. *The same five-layer measurement on further standard benchmarks.* Done for BFCL v4 and for τ -bench **retail** (§4.1), so the measurement-validity concern is *not* specific to our harness, and it now covers one interactive suite with a simulated user and stateful environment. What the τ -bench arm does *not* establish is an outcome effect: 7 versus 10 tasks solved on three discordant pairs is far from significant, and its user simulator is a local Llama-3.1-8B rather than **gpt-4o**, so its absolute scores are not comparable to the published leaderboard. Suites whose tool schemas are much larger, or whose scoring depends on argument-level matching, remain untested.

I Statistical estimands and retrospective test family

I.1 Finite-sample interval for the formal RL endpoint

Let p_+ denote the probability that an evaluation item passes only under repaired FC and p_- the probability that it passes only under broken FC. The paired risk difference is $\Delta = p_+ - p_-$. At step 150, the counts are $(n_+, n_-, n) = (0, 3, 542)$. For each discordance probability we invert the binomial distribution to obtain a two-sided 97.5% Clopper–Pearson interval, denoted $[L_+, U_+]$ and $[L_-, U_-]$. The union bound gives at least 95% coverage for $[L_+ - U_-, U_+ - L_-]$ under independent evaluation-item pairs. This conservative construction does not rely on a normal approximation with three discordant pairs. The point difference is -0.55 percentage points and the interval is $[-1.79, 0.72]$ percentage points. For repaired step 0 versus 150, $(1, 4, 542)$ gives $[-2.06, 1.01]$ percentage points. These are unadjusted endpoint intervals, not simultaneous intervals for the whole test family. They quantify item-pair uncertainty under an independence assumption, not training-seed, inference-retest, benchmark-selection or task-cluster uncertainty.

I.2 Explicit reporting family

This is a retrospective reporting audit, not a preregistered multiplicity policy. The family includes all 15 metrics in the five audited Llama control contrasts, even when only one metric is highlighted,

plus the 12 distinct comparisons below. Repeated appearances of the same contrast count once. Nominal p values are exact two-sided McNemar tests; adjusted $p = \min(1, 27p)$, with threshold $0.05/27 = 0.00185185$. Historical 1.5B motivating runs and abandoned/invalid runs are not confirmatory evidence and are outside this family; their original disclosures remain in the historical audit. The two both-parsed contrasts are retained in the family but remain post-treatment descriptive comparisons, not causal effects. This explicit family replaces the earlier approximate count of twelve; in particular, the Llama strict-to-ReAct and both-parsed contrasts no longer clear the corrected threshold.

Table 33: Retrospective test inventory. G/L are later-arm gains/losses in the named binary metric; for wrong-tool metrics a gain means more wrong-tool calls, not an improvement. A star marks $p \leq 0.05/27$.

ID	Contrast and metric	n	G/L	nominal p
T01	Llama terse to rich: wrong_tool	100	5/6	1.00
T02	Llama terse to rich: turn1_pass	100	4/6	0.754
T03	Llama terse to rich: final_pass	100	10/10	1.00
T04	Llama terse to thought: wrong_tool	100	37/1	2.84e-10*
T05	Llama terse to thought: turn1_pass	100	1/23	0.00000298*
T06	Llama terse to thought: final_pass	100	4/29	0.0000109*
T07	Llama terse to official: wrong_tool	100	0/1	1.00
T08	Llama terse to official: turn1_pass	100	0/1	1.00
T09	Llama terse to official: final_pass	100	1/6	0.125
T10	Llama terse to strict: wrong_tool	100	0/23	2.38e-7*
T11	Llama terse to strict: turn1_pass	100	13/1	0.00183*
T12	Llama terse to strict: final_pass	100	15/3	0.00754
T13	Llama official to strict: wrong_tool	100	0/22	4.77e-7*
T14	Llama official to strict: turn1_pass	100	14/1	0.000977*
T15	Llama official to strict: final_pass	100	18/1	0.0000763*
T16	Llama strict to ReAct final	100	27/8	0.00188
T17	Llama terse to ReAct final	100	38/7	0.00000312*
T18	Qwen Hermes to adapter final	100	16/7	0.0931
T19	Qwen adapter to ReAct final	100	17/5	0.0169
T20	Qwen Hermes to adapter final (random)	300	43/25	0.0385
T21	Qwen adapter to ReAct final (random)	300	59/18	0.00000306*
T22	Llama strict to ReAct final (random)	300	51/31	0.0352
T23	Qwen both-parsed FC to ReAct final	83	11/4	0.118
T24	Llama both-parsed FC to ReAct final	96	25/7	0.00210
T25	tau documented to repaired success	115	3/0	0.250
T26	P3 broken to repaired final at step150	542	0/3	0.250
T27	P3 repaired step0 to step150 final	542	1/4	0.375

The accompanying `submission_statistics.py` recomputes the 15 Llama tests from item-level records and the remaining tests from the explicit paired counts, with their sources identified in `submission_statistics.json`. Thus nominal and adjusted values can be reproduced without GPUs. The latter count-based recomputation is not a new item-level audit of every historical comparison.

The later post-hoc native FC evaluation adds a separately identified nine-test exploratory family (Appendix G.15): three paired weight contrasts crossed with final success, tool initiation and rescue. Its per-item inputs, nominal and Bonferroni-adjusted values are supplied separately. No new contrast is nominally significant, and none becomes significant if all 36 tests are pooled. The historical 27-test audit is preserved, not retroactively called preregistered.